Seminario de Práctica de Informática

Trabajo Práctico N°2

Módulo 2

Código: INF275-11987

Comisión de la materia: CAT A - EDH

Alumno: Valentino Belli

DNI: 46497315

Legajo: VINF016781

Titular experto: Pablo Alejandro Virgolini

Titular disciplinar: Pablo Alejandro Virgolini

Fecha de entrega: 28/09/2026

Índice

Consignas 3

Formato Entregable 4

Desarrollo 5

1- Etapa de análisis 5

2- Etapa de diseño 19

3- Etapa de implementación 19

4- Etapa de pruebas 21

4.1 Plan de pruebas 21

4.2 Modelo de pruebas 22

4.3 Tratamiento de defectos 25

4.4 Evaluación de pruebas 26

5- Prototipo de interfaz 27

6- Definición de base de datos para el sistema 28

7- Diagrama entidad-relación de la base de datos 29

8- Creación de las tablas MySQL 30

9- Inserción, consulta y borrado de registro. 34

10- Definiciones de comunicación 40

Consignas

Considera que continúas trabajando en la organización cuyo problema definiste resolver con un proyecto informático, y estás en el equipo de desarrollo que va a llevar adelante el mismo.

Esta segunda actividad te permitirá aplicar de forma completa los modelos de análisis, diseño, implementación y pruebas del PUD. Es fundamental asegurar la trazabilidad del sistema, seleccionando adecuadamente los artefactos a presentar en cada etapa, utilizando correctamente el lenguaje unificado de desarrollo (UML) para representar y comunicar los diferentes aspectos del sistema.

A continuación, deberás diseñar un modelo de datos relacional que represente las entidades, atributos y relaciones específicas del sistema. Es importante aplicar los principios de normalización para asegurar que las tablas en la base de datos estén estructuradas de manera eficiente y plantear la realización de un diagrama entidad-relación. Recuerda asegurar que las estructuras de datos estén alineadas con el diseño del sistema y cumplan con los requisitos de integridad y rendimiento.

Finalmente, podrás definir cuáles son los protocolos y estándares adecuados para la comunicación entre el sistema y otros sistemas externos o la base de datos: entorno de red, infraestructura física, control de enlace de datos, etc.

Los entregables a presentar son los siguientes:

- Etapa de análisis.
- Etapa de diseño.
- Etapa de implementación.
- Etapa de pruebas.
- Definición de base de datos para el sistema.
- Diagrama entidad-relación de la base de datos.
- Creación de las tablas MySQL.
- Inserción, consulta y borrado de registros.
- Presentación de las consultas SQL.
- Definiciones de comunicación.

Formato Entregable

La actividad propuesta debe ser elaborada en forma individual y respetar los criterios de presentación mencionados a continuación. Se debe realizar una entrega incremental, considerando las correcciones realizadas en la entrega previa.

El informe se debe presentar en un documento en formato PDF (papel A4, letra Calibri 11, espaciado simple). Debe incluir las respuestas a las consignas de forma prolija y ordenada, con su correspondiente explicación para justificar las decisiones adoptadas. Incluir una portada y respetar el siguiente criterio para el nombre del archivo:

«APELLIDO-NOMBRE»-AP2.PDF

La realización de todos los diagramas requiere utilizar una herramienta de libre elección, pero asegurando la prolijidad y claridad de presentación. Todos los diagramas deben incluirse en el documento a entregar con su correspondiente explicación.

Para la persistencia de los datos, deberás utilizar una base de datos MySQL, y para el desarrollo del prototipo, una IDE de Java. La explicación debe estar en el documento a entregar e incluir un enlace a la versión completa (código y archivos asociados) en Github: https://github.com/.

Se evaluará la precisión en las respuestas, capacidad de síntesis, redacción, ortografía y calidad de la presentación.

En caso de requerirlo, es importante revisar el cumplimiento de las normas de estilo American Psychological Association (APA).

Desarrollo

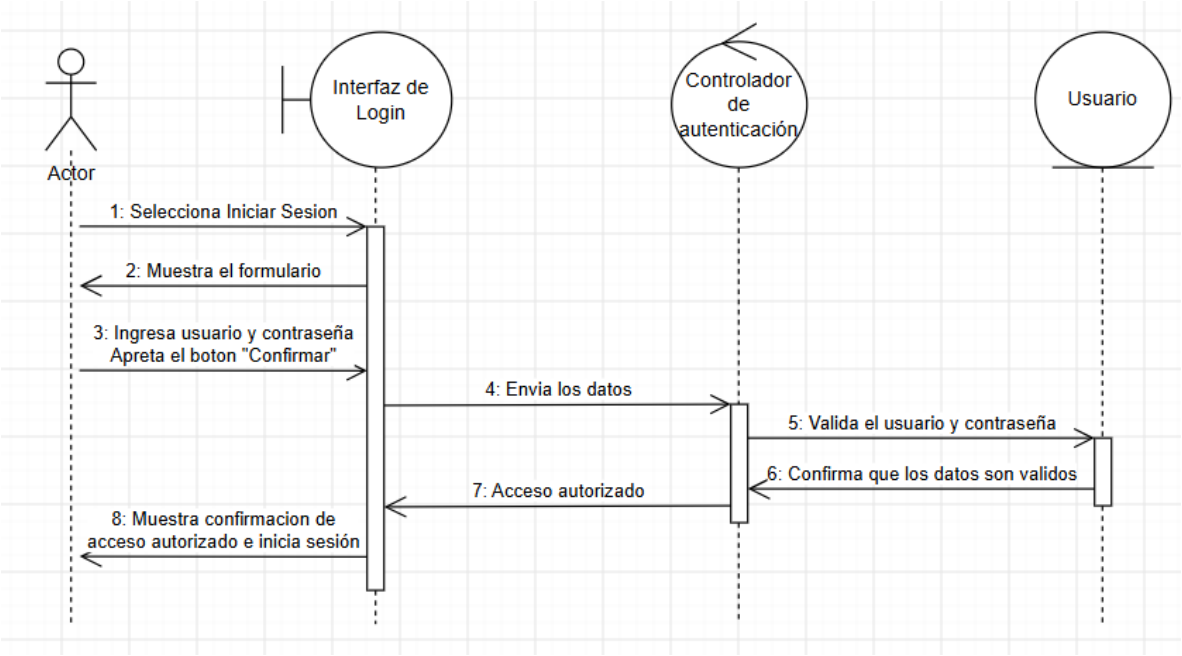
Github del proyecto: https://github.com/Valentino-Belli/BELLI-VALENTINO-AP2

1- Etapa de análisis

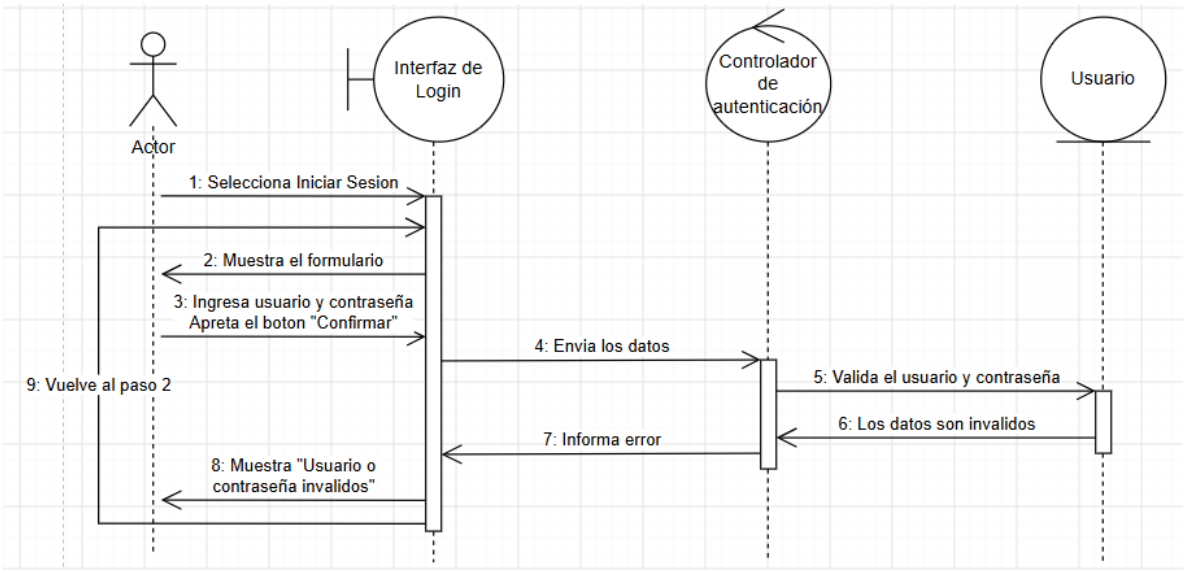
Para empezar con el análisis primero elegimos los casos de uso, luego revisamos los flujos normales y alternativos, y hacemos el diagrama de secuencia. Una vez finalizados los diagramas de secuencia de los casos de uso elegidos, se identifican las diferentes clases que aparecen para luego construir el diagrama de clases del análisis. Por último, se organizan esas clases en paquetes.

Caso de uso: 001 – Iniciar sesión

Flujo normal:

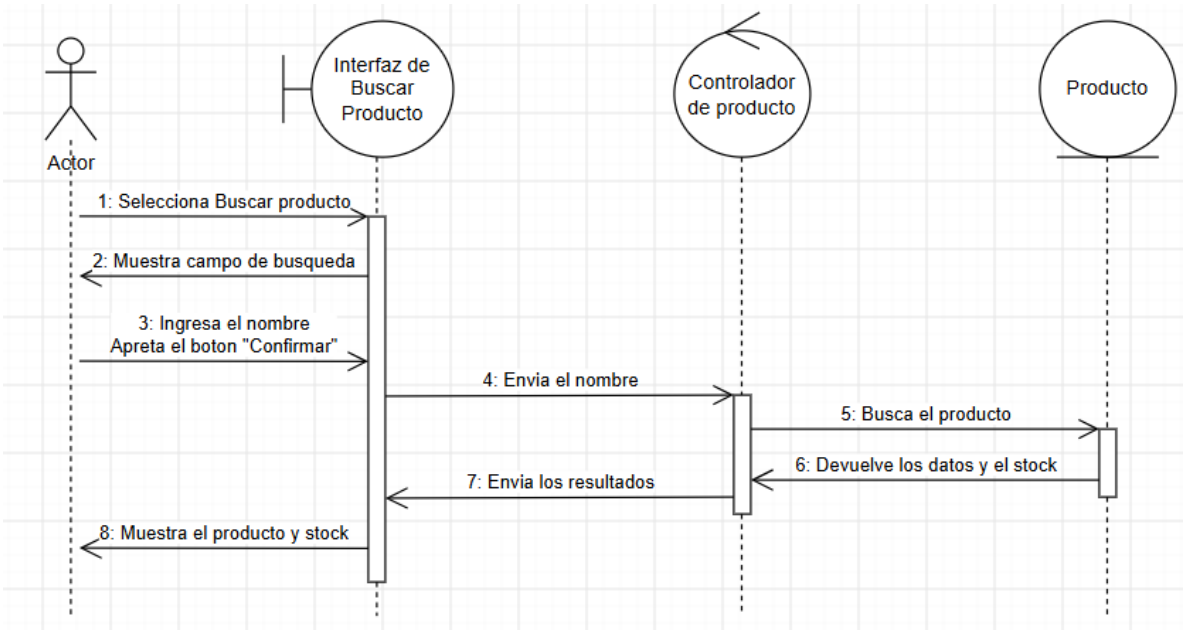


Flujo alternativo: Validación de datos de usuario fallida

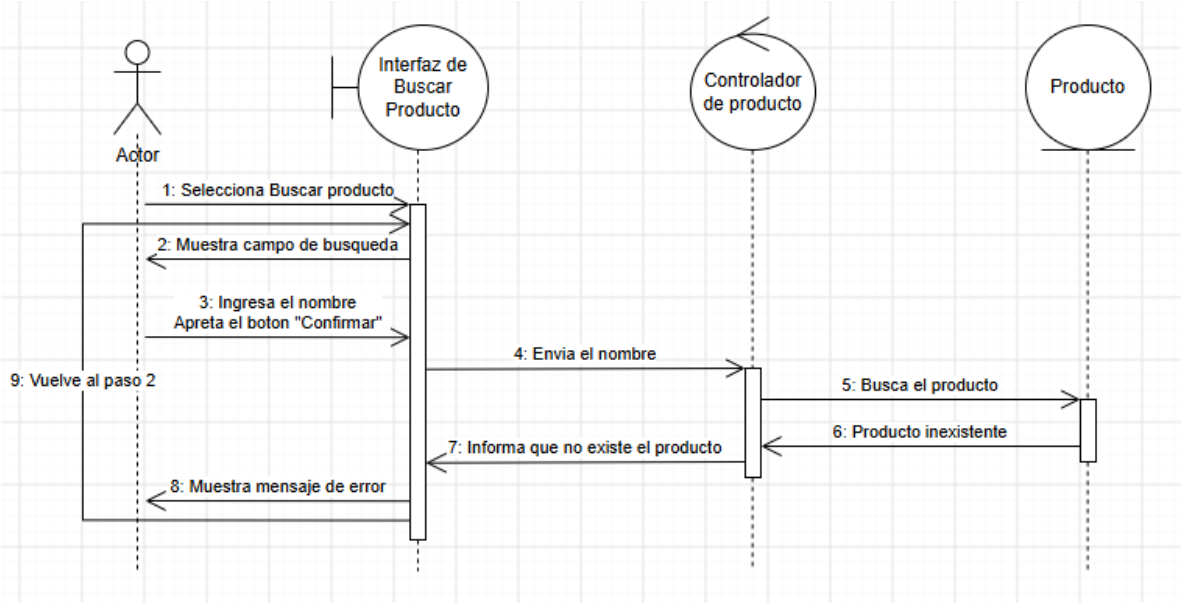


Caso de uso: 002 – Buscar producto

Flujo normal:

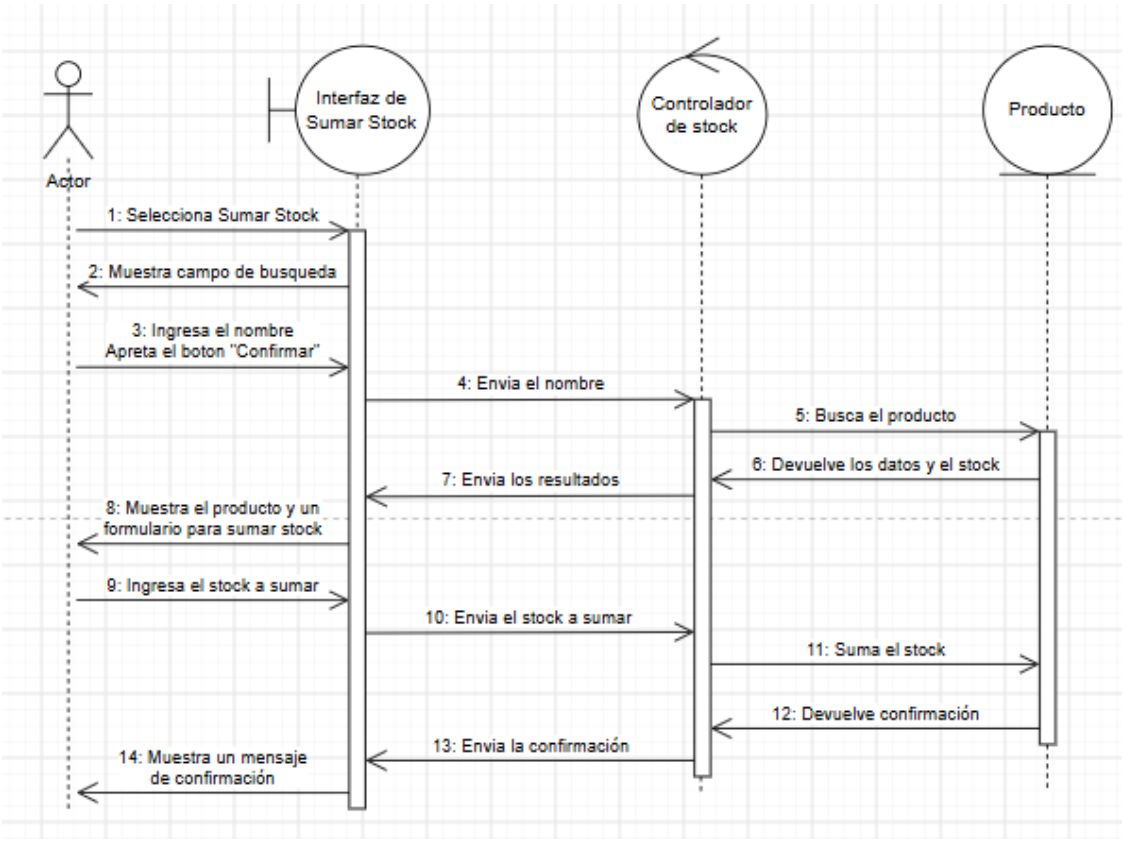


Flujo alternativo: Producto no encontrado

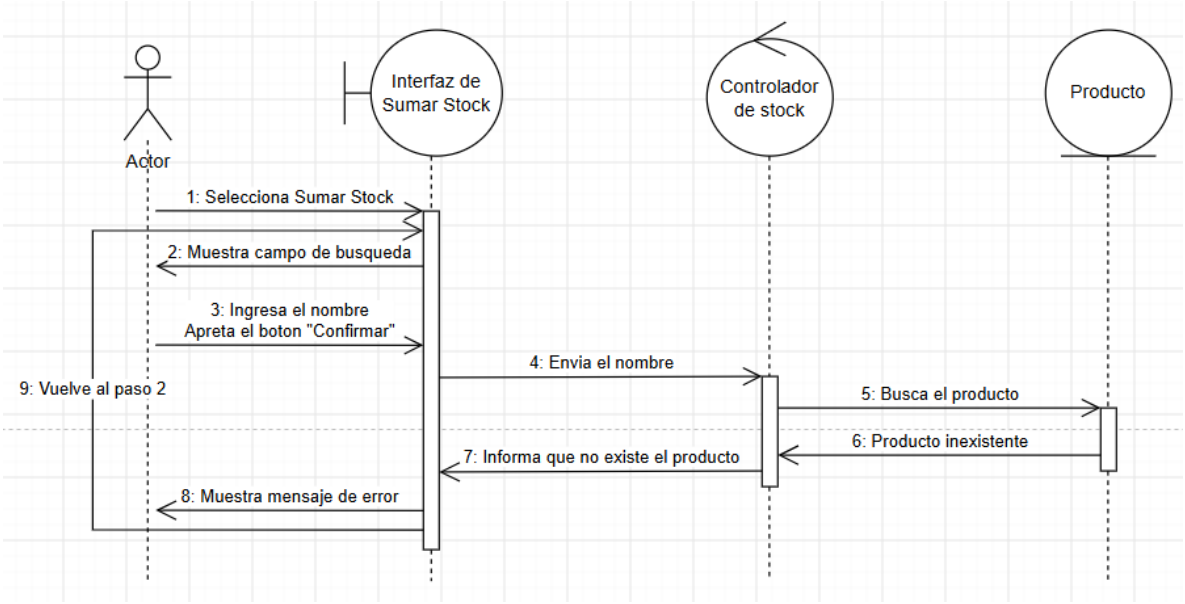


Caso de uso: 003 – Sumar stock

Flujo normal:

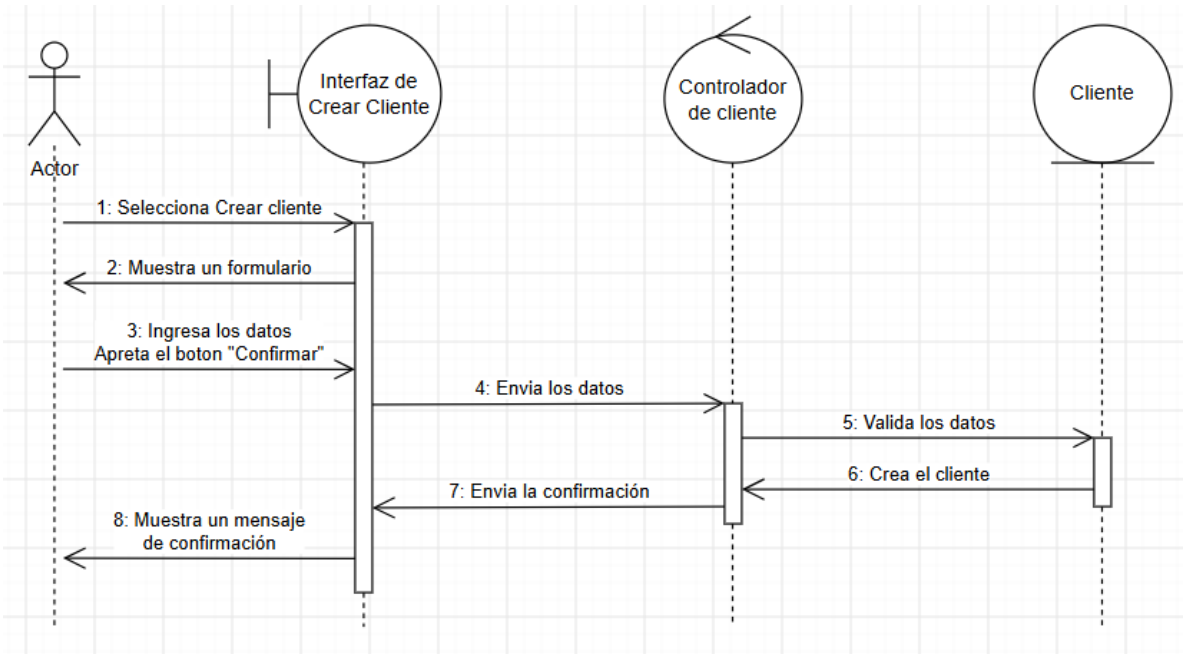


Flujo alternativo: Producto no encontrado

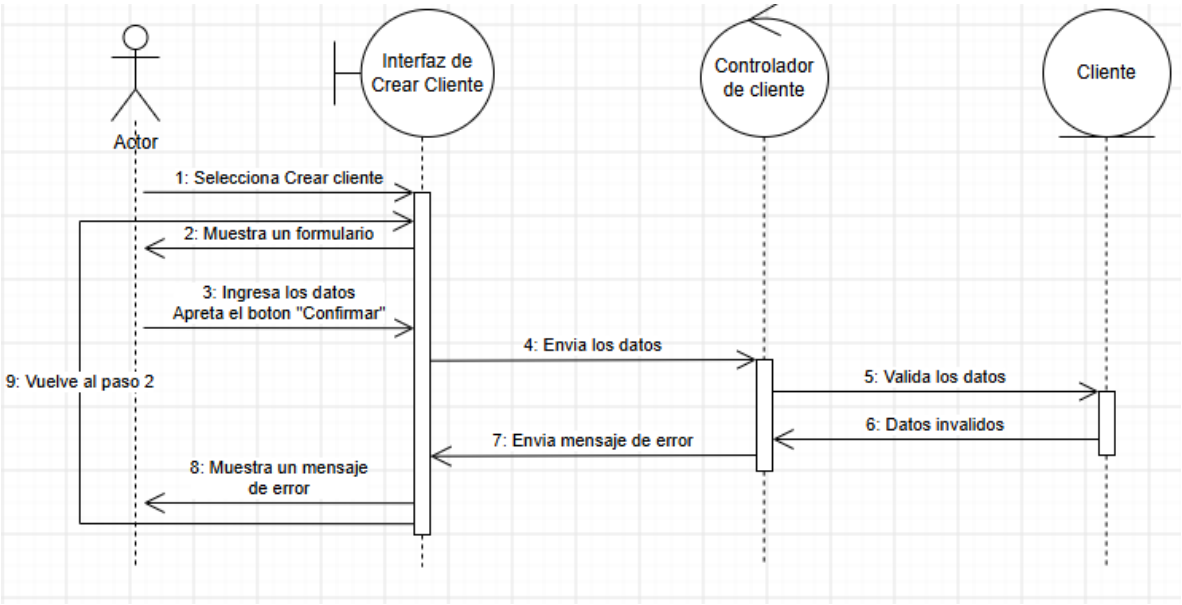


Caso de uso: 004 – Crear cliente

Flujo normal:

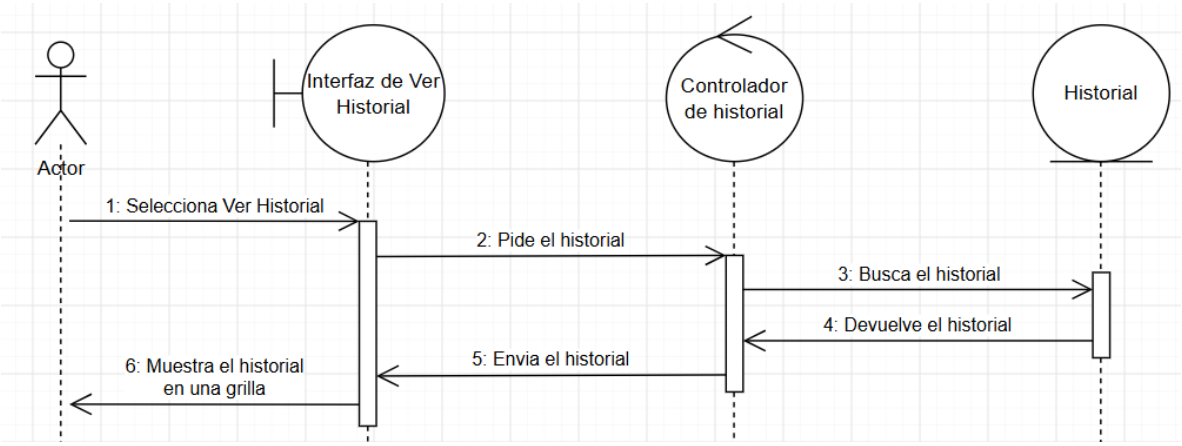


Flujo alternativo: Datos inválidos



Caso de uso: 005 – Ver Historial

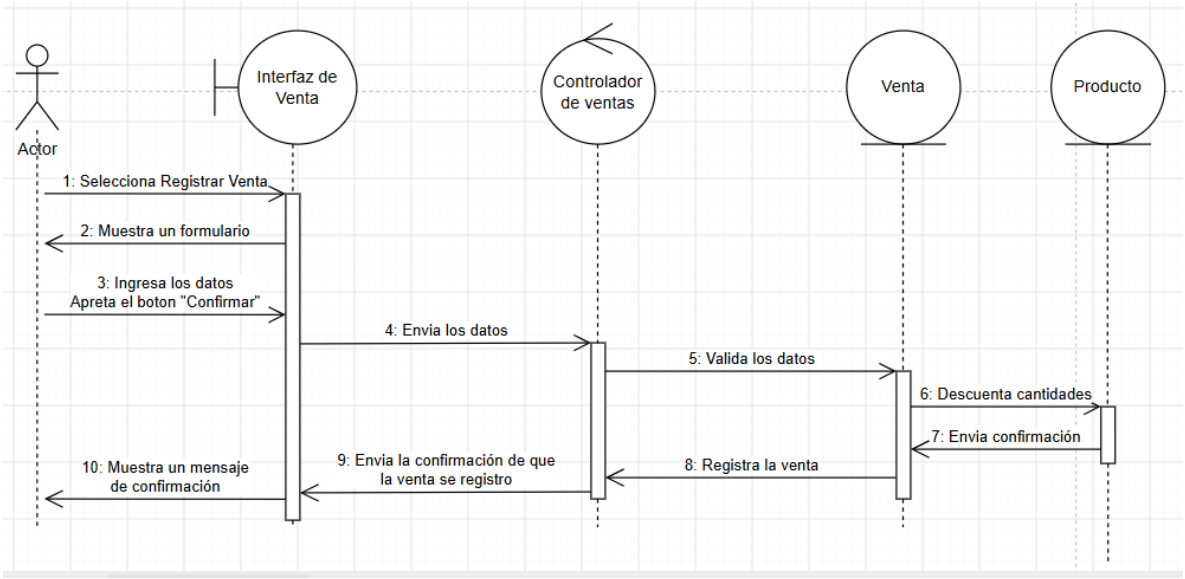
Flujo normal:



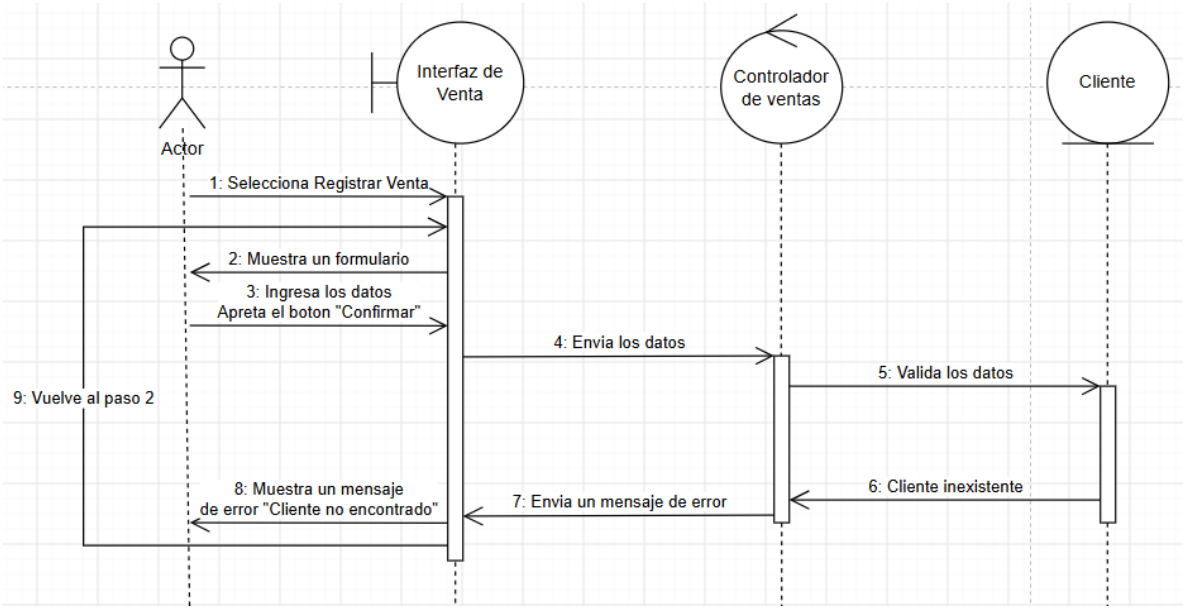
Flujo alternativo: No contempla un flujo alternativo

Caso de uso: 006 – Registrar venta a cliente

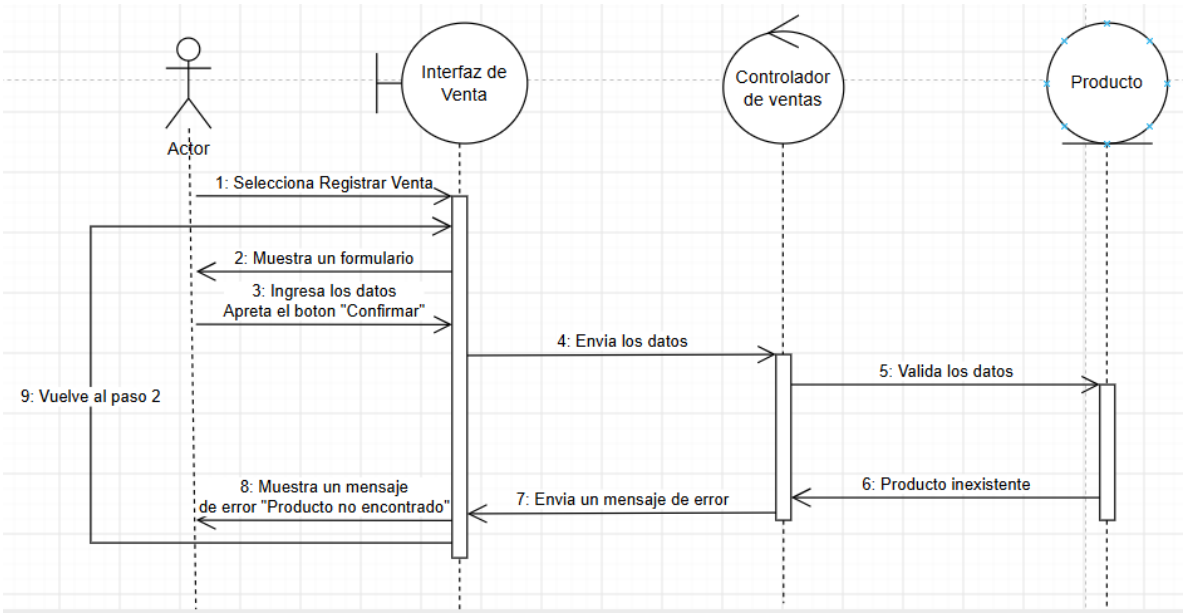
Flujo normal:



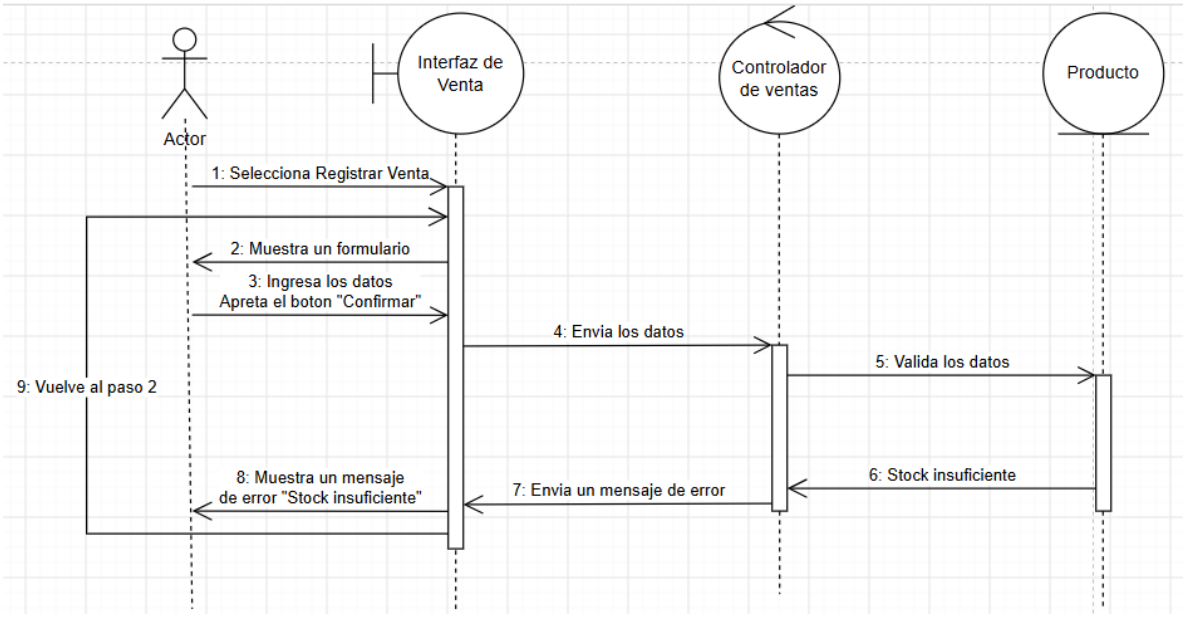
Flujo alternativo: Cliente no encontrado



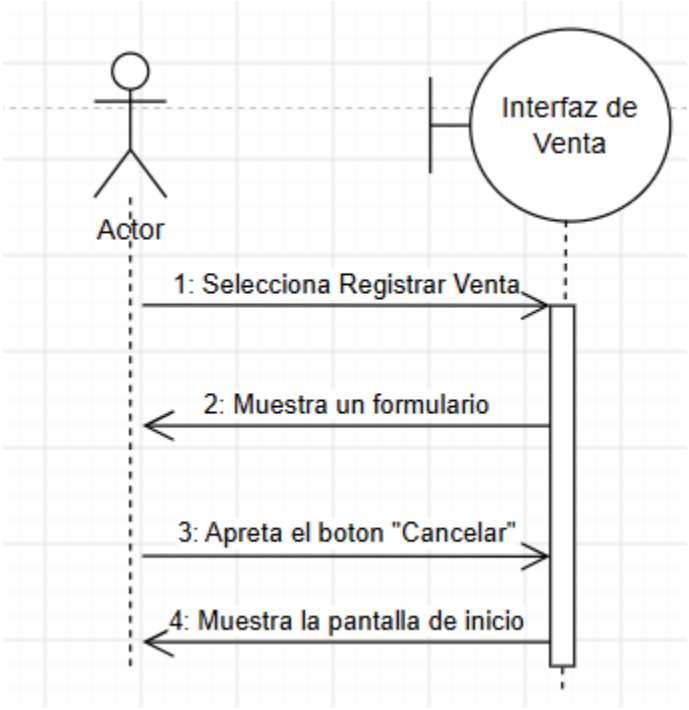
Flujo alternativo: Producto no encontrado



Flujo alternativo: Stock insuficiente

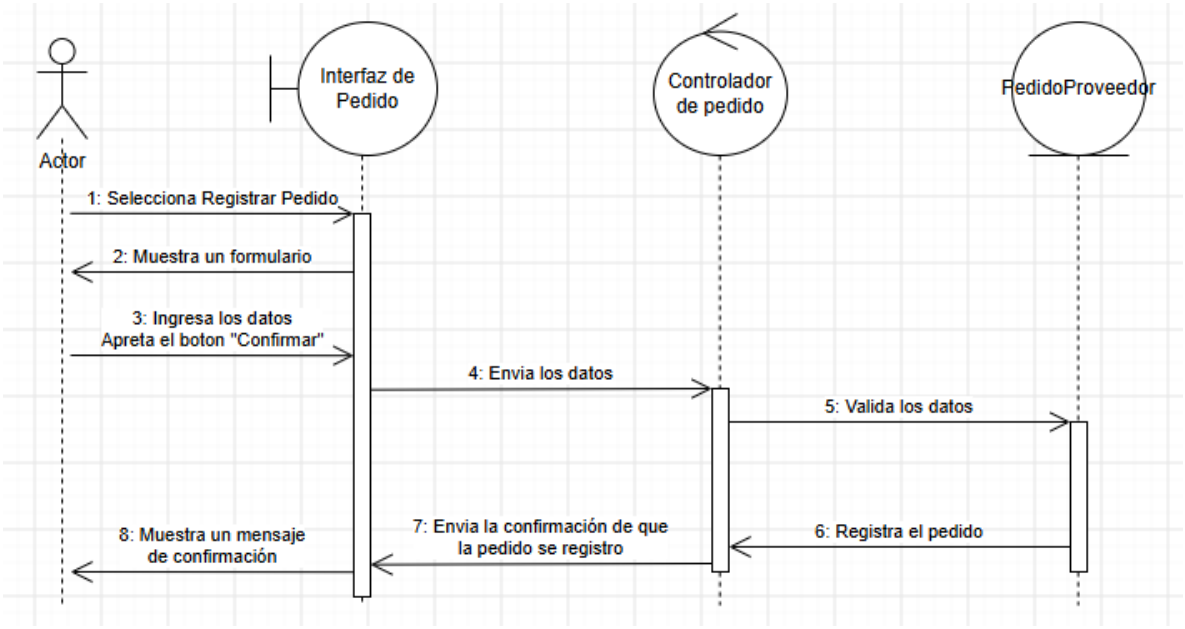


Flujo alternativo: Cancelación de venta

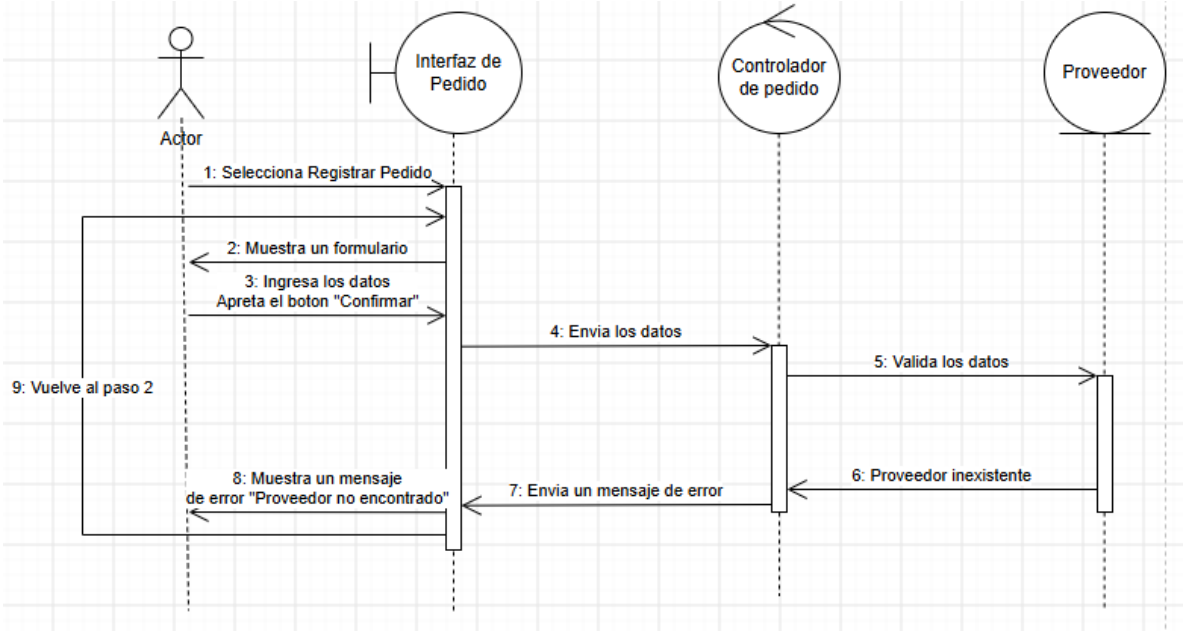


Caso de uso: 007 – Registrar pedido proveedor

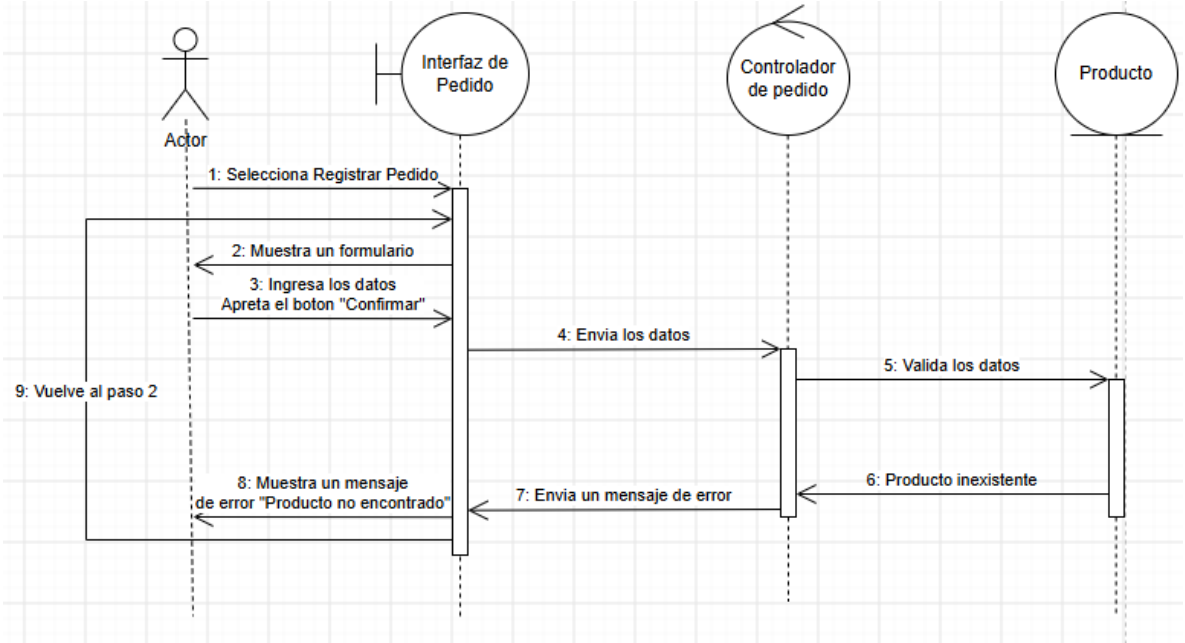
Flujo normal:



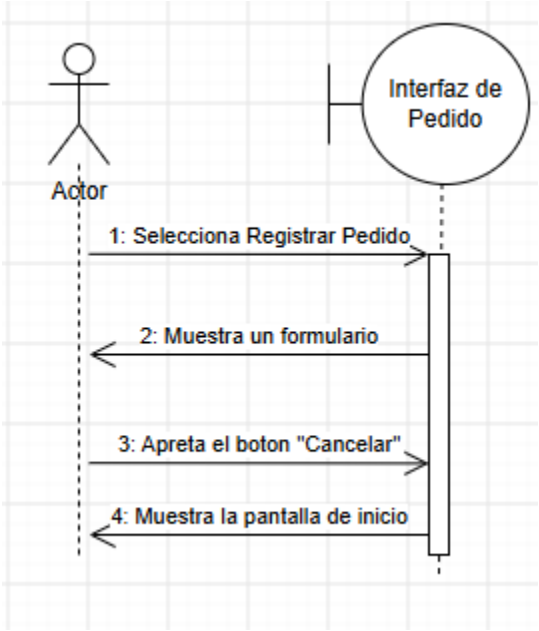
Flujo alternativo: Proveedor no encontrado



Flujo alternativo: Producto no encontrado

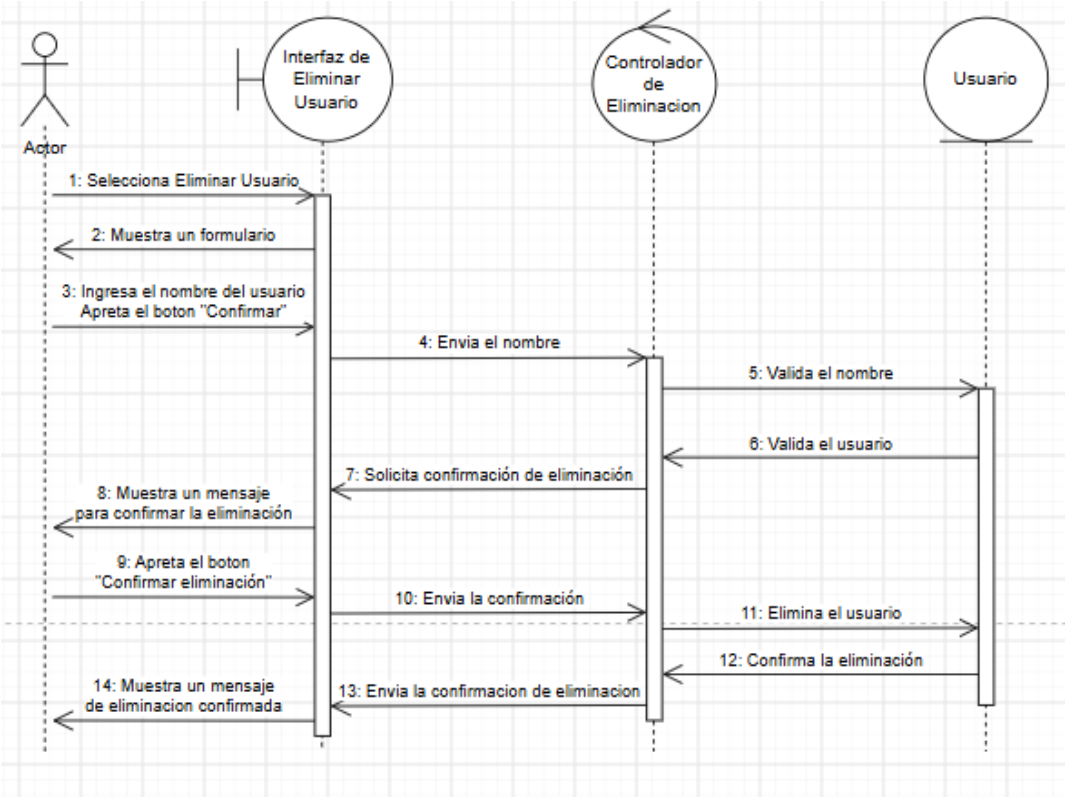


Flujo alternativo: Cancelación de pedido

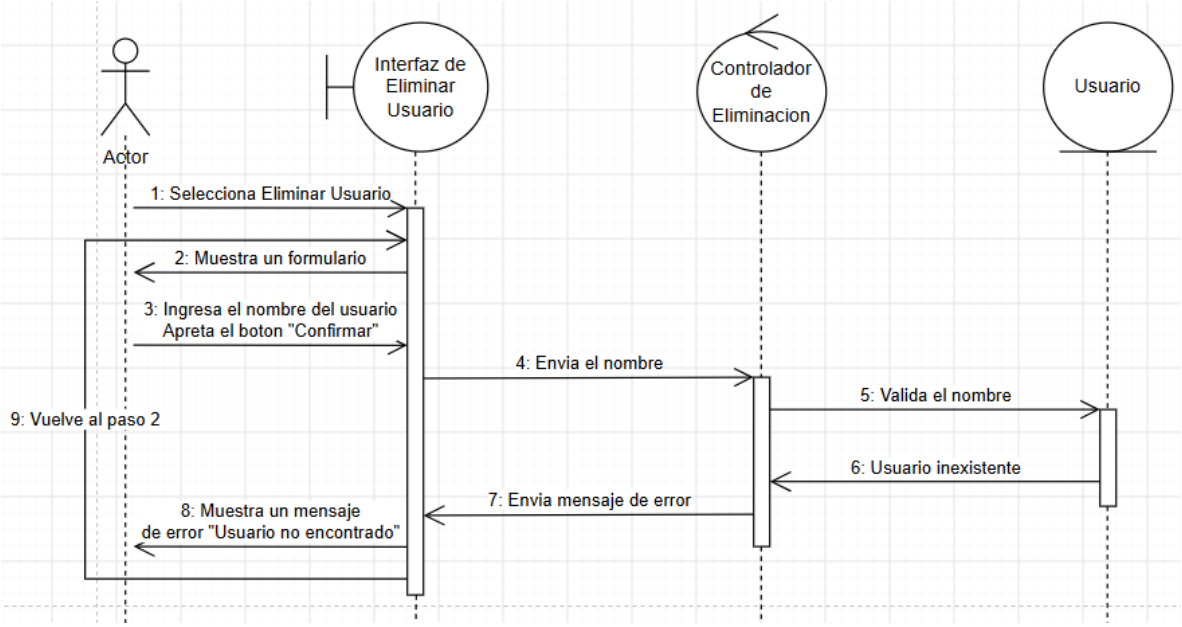


Caso de uso: 008 – Eliminar usuario

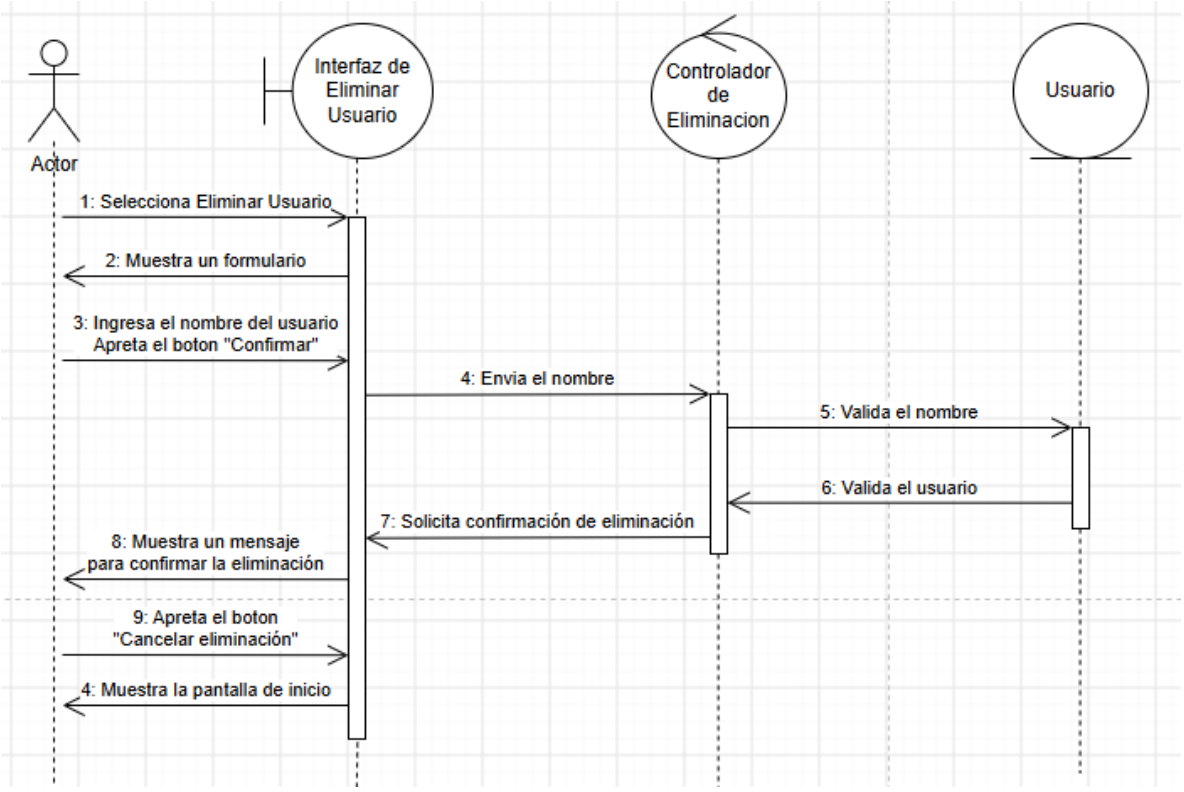
Flujo normal:

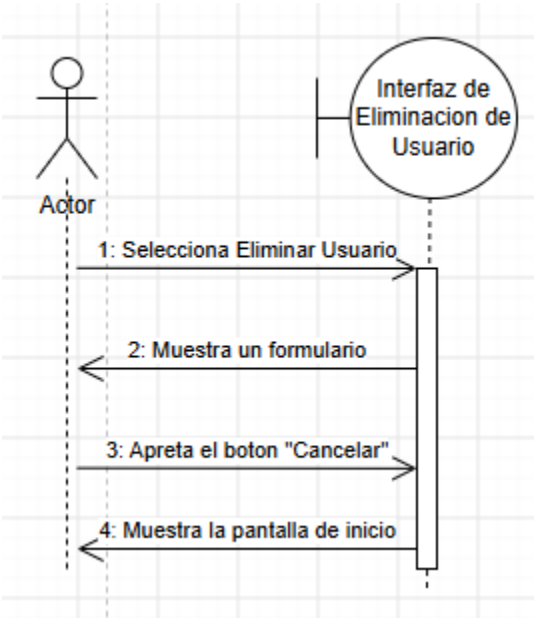


Flujo alternativo: Usuario no encontrado



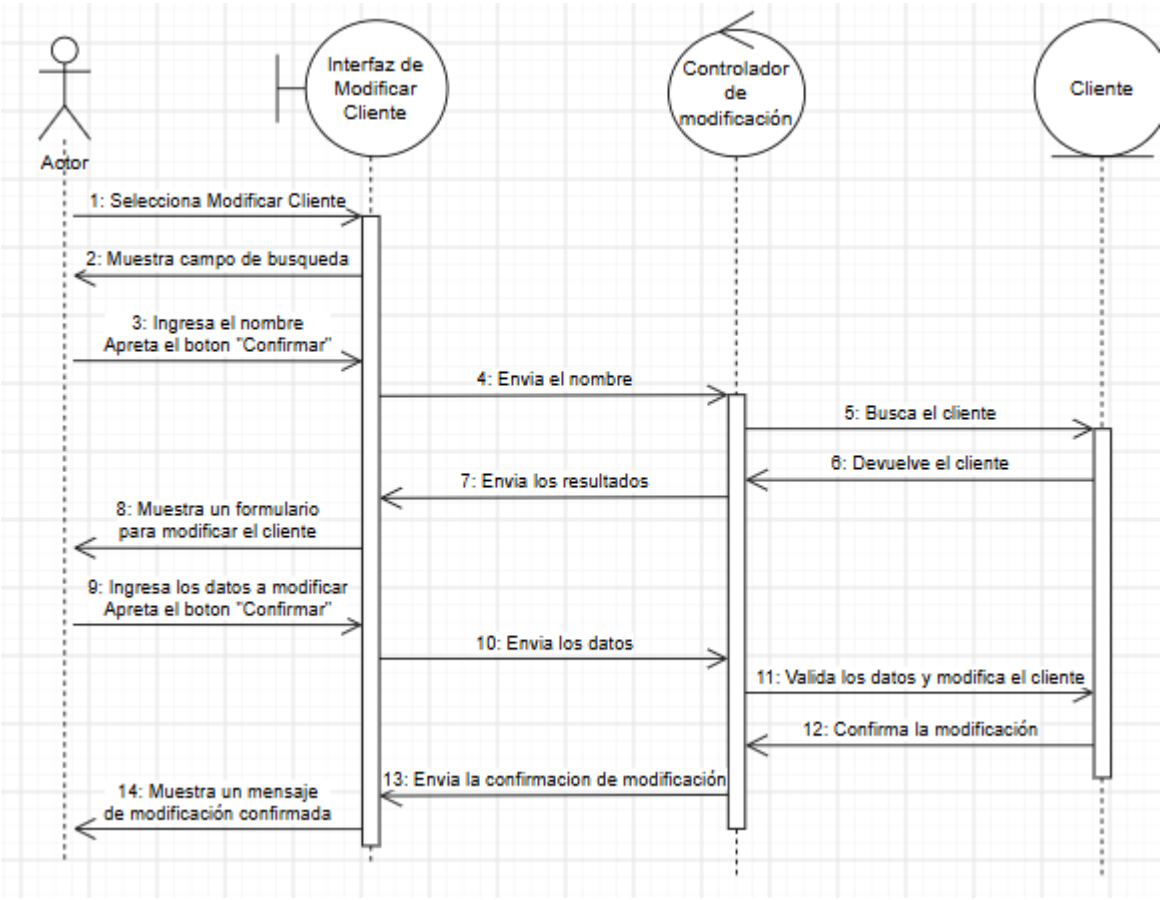
Flujo alternativo: Cancelación de eliminación



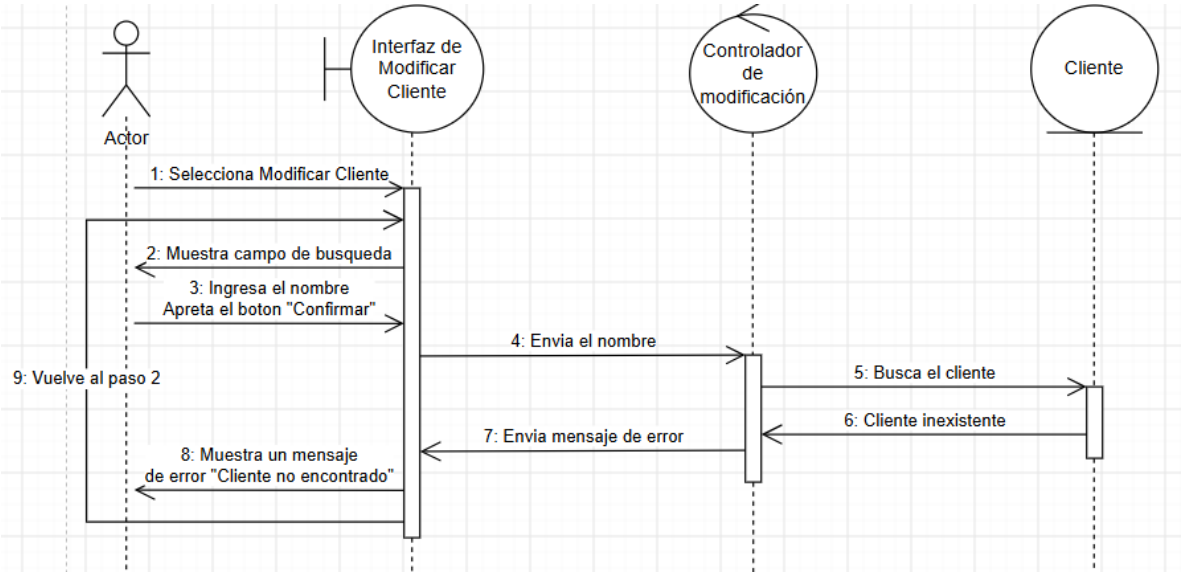


Caso de uso: 009 – Modificar Cliente

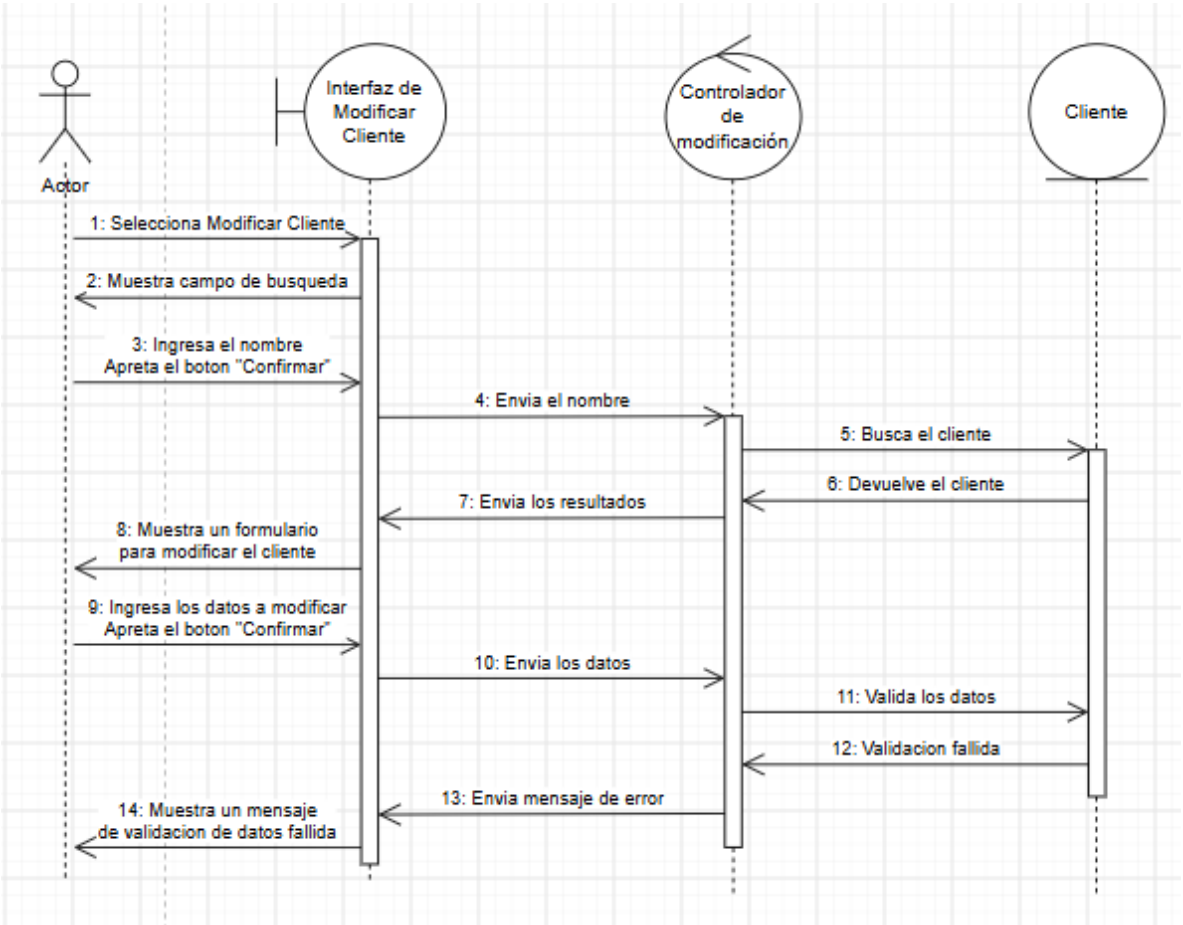
Flujo normal:



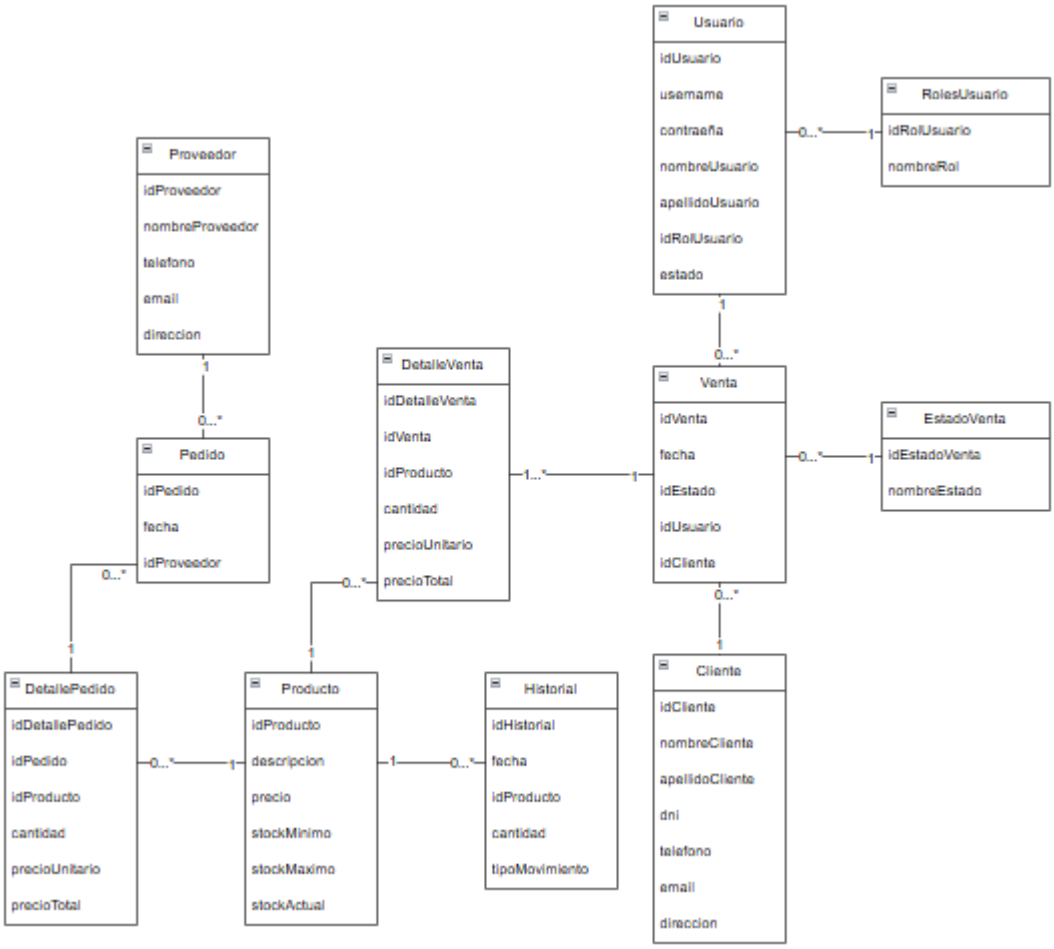
Flujo alternativo: Cliente no encontrado



Flujo alternativo: Validación de datos fallida



El diagrama de clases de análisis es el siguiente:



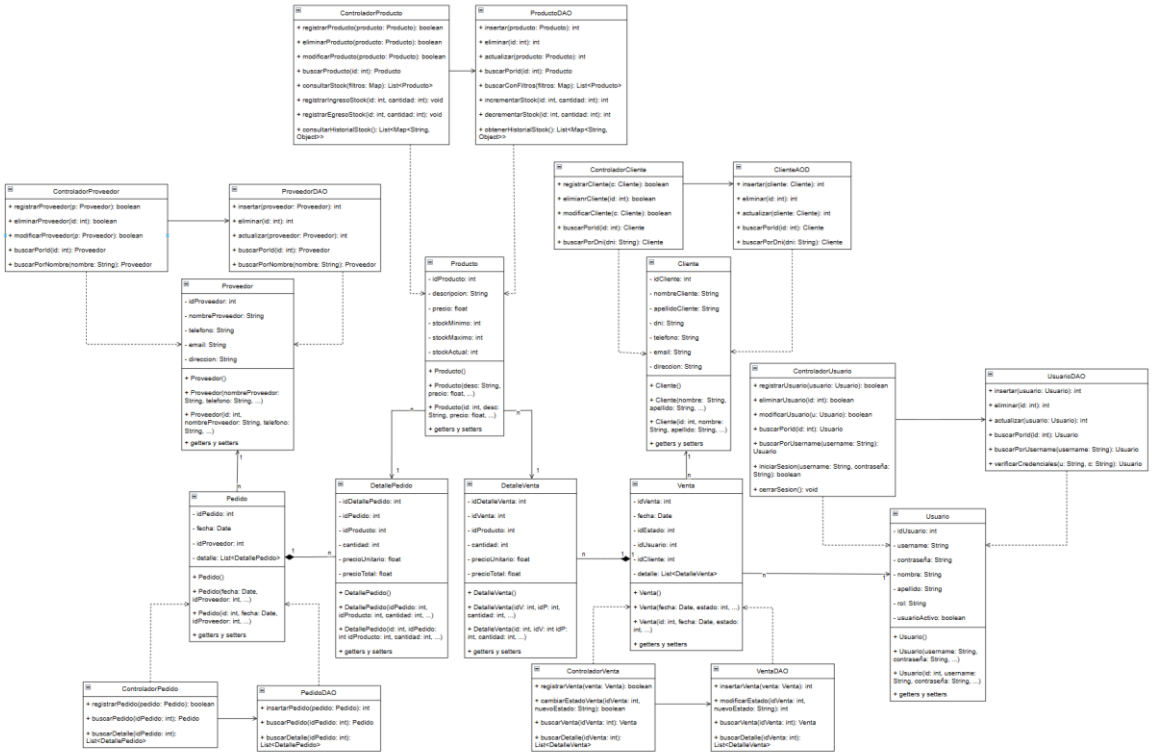
Los distintos paquetes de análisis son los siguientes:

Paquete	Clases	Descripción
Paquete de seguridad y acceso	Usuario – RolesUsuario	Gestiona la autenticación y los permisos de los usuarios del sistema.
Paquete de ventas	Cliente – Venta – DetalleVenta - EstadoPedido	Administra todo lo relacionado con las ventas de productos y registro de clientes.
Paquete de inventario	Producto – Historial	Controla las existencias de mercadería y los movimientos de almacén.

Paquete de compras	Proveedor – Pedido - DetallePedido	Se encarga del abastecimiento de productos a través de los proveedores oficiales.

2- Etapa de diseño

Para la etapa de diseño debemos hacer el diagrama de clases de diseño, el cual es similar al diagrama de análisis, pero agrega detalles técnicos como tipos de datos y métodos, debido a que está más enfocado en la implementación y desarrollo del sistema.



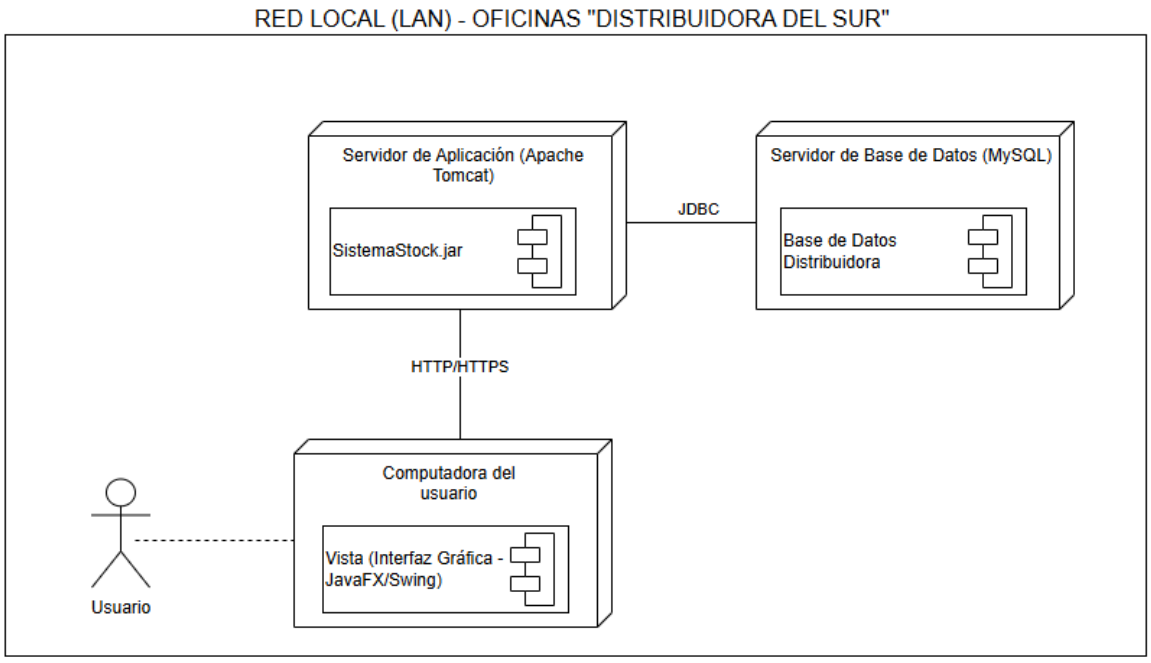
3- Etapa de implementación

Los requerimientos no funcionales (RNF) del sistema son los siguientes.

ID	Descripción
RNF01	El sistema debe estar desarrollado en Java
RNF02	El sistema debe contar con una base de datos MySQL.

RNF03	El sistema debe tener una disponibilidad de 7 x 24, con un tiempo de inactividad de un 1 % en el mes para mantenimiento.
RNF04	Para la persistencia y consulta de datos en la BD se debe utilizar un patrón MVC (modelo-vista-controlador).
RNF05	El sistema debe estar instalado en una infraestructura propia en las oficinas de “Distribuidora del Sur”
RNF06	El sistema debe realizar copias de seguridad (backups) automáticas de la base de datos diariamente fuera del horario comercial
RNF07	Los servidores de la infraestructura propia deben contar con un sistema de energía ininterrumpida (UPS/SAD) para prevenir pérdidas de datos ante cortes eléctricos
RNF08	El sistema debe soportar al menos 15 usuarios operando en simultáneo (administradores, operarios de depósito y facturistas) sin degradar el tiempo de respuesta

En base a los RNF, el diagrama de despliegue del sistema queda de la siguiente manera:



4- Etapa de pruebas

4.1 Plan de pruebas

Para empezar con la etapa de pruebas, se presenta el plan de pruebas básico propuesto para el caso de uso CU001, “Iniciar Sesión. Se va a utilizar un código de prueba según la siguiente nomenclatura.

- CP: Caso de prueba.
- Tercera letra: Tipo de prueba.
- Cuarta y quinta letra: Número de prueba.

Caso de uso	Código de prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU001	CPC01	Componente	Cobertura de caminos mínimos	Aplicación de caja blanca al método iniciarSesion(nombre, contraseña) de la clase ControladorUsuario para asegurar que el camino del true (credenciales válidas) y del false (clave errónea o usuario inactivo/inexistente) respondan correctamente.
CU001	CPF02	Componente	Partición de equivalencia / Valores límite.	Aplicación de caja negra al método de autenticación ingresando datos válidos, datos inexistentes, y campos vacíos para comprobar el manejo seguro de excepciones en Java.
CU001	CPS03	Sistema	Pruebas funcionales de interfaz.	Verificación del comportamiento del flujo completo desde la pantalla gráfica. Se espera que al oprimir "Ingresar" con credenciales correctas se abra el menú principal y que ante un error se bloquee el acceso y se muestre un mensaje de alerta al operario.

4.2 Modelo de pruebas

Lo primero que se hace es definir las clases de equivalencia y valores frontera para las entradas.

Campo: Nombre de usuario (Longitud permitida: 4 a 20 caracteres)

Equivalencias validas	
CE1	Cadena de texto con longitud entre 4 y 20 caracteres existentes en la BD.

Equivalencias invalidas	
CE2	Campo vacío (longitud 0).
CE3	Cadena fuera de los límites permitidos (menor a 4 o mayor a 20 caracteres).
CE4	Nombre de usuario que no existe en la base de datos de la distribuidora.

Campo: Contraseña (Longitud permitida: 6 a 15 caracteres)

Equivalencias validas	
CE5	Contraseña correcta que coincide con el usuario en la BD.

Equivalencias invalidas	
CE6	Campo vacío (longitud 0).
CE7	Contraseña incorrecta para este usuario
CE8	Cadena fuera de los límites permitidos (menor a 6 o mayor a 15 caracteres).

Clase/Frontera	Datos de entrada (Usuario y contraseña)	Comportamiento esperado	Mensaje emitido por el sistema
CE1 + CE5 (Éxito)	Admin / cLAvE132	Permite el acceso. Cierra el login y abre el inicio del programa	“Inicio de sesión exitoso. Bienvenido al sistema.”

CE2 (Campo “username” vacío)	[Vacío] / cLAvE132	Bloquea el envío. Foco en “username”	“Error: El campo 'Nombre de usuario' no puede estar vacío."
CE6 (Campo “contraseña” vacío)	Admin / [Vacío]	Bloquea el envío. Foco en “contraseña”	“Error: El campo 'Contraseña’ no puede estar vacío.”
CE3 (Frontera inferior)	Adm / cLAvE132	Rechazado en el cliente por el formato	“Error: El usuario debe tener entre 4 y 20 caracteres.”
CE3 (Frontera superior)	AdministradorDelSistema / cLAvE132	Rechazado en el cliente por el formato	“Error: El usuario debe tener entre 4 y 20 caracteres.”
CE4 (Usuario inexistente)	usuarioFalso / cLAvE132	Falla la autenticación en el controlador.	"Error: Usuario o contraseña incorrectos."
CE7 (Contraseña incorrecta)	Admin / ClaVe978	Falla la autenticación en el controlador.	"Error: Usuario o contraseña incorrectos."
CE8 (Frontera inferior)	Admin / cLAvE	Rechazado en el cliente por el formato	“Error: La contraseña debe tener entre 6 y 15 caracteres.”
CE8 (Frontera superior)	Admin / cLAvEsEgUrA132465	Rechazado en el cliente por el formato	“Error: La contraseña debe tener entre 6 y 15 caracteres.”

Algunos de los casos de prueba para el caso de uso C001 “Iniciar Sesión” son los siguientes:

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-001	Funcional	Verificar que un operario activo con credenciales correctas pueda ingresar al sistema.	El usuario operario1 debe estar registrado en la base de datos y el valor usuarioActivo = true.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	Usuario: operario1 Contraseña: Stock123	Acceder al sistema. Ingresar los datos en sus respectivos campos. Presionar “Ingresar”	El método retorna true, la pantalla de login se cierra y se despliega el menú principal.

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-002	Funcional	Verificar que el sistema deniegue el acceso ante una contraseña inválida.	El usuario operario1 debe estar registrado en la base de datos y el valor usuarioActivo = true.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	Usuario: operario1 Contraseña: ClaveErronea	Acceder al sistema. Ingresar los datos en sus respectivos campos. Presionar “Ingresar”	El método retorna false, permanece en la pantalla de login y se muestra una ventana emergente con el mensaje de la clase de equivalencia CE7.

4.3 Tratamiento de defectos

Este artefacto registra las fallas encontradas durante la ejecución de las pruebas. Algunos ejemplos de posibles defectos que se podrían encontrar en un caso de uso “Iniciar Sesión” son estas:

Defecto ID	DEF-CU001-01
Caso de Prueba Asociado	CP-002 (Intento de inicio de sesión con Contraseña Incorrecta).
Descripción del Defecto	Al ingresar una contraseña incorrecta, el sistema lanza una excepción en la consola de Java (NullPointerException en ControladorUsuario.java:42) en lugar de mostrar la alerta gráfica. La pantalla se queda congelada y el operario no sabe qué pasó.
Severidad	Alta (Bloquea la operación normal de manejo de errores del sistema).
Prioridad de Resolución	Alta
Estado	Corregido
Tratamiento / Resolución	Se añadió un bloque try-catch en el método iniciarSesion del controlador para capturar la respuesta nula de la base de datos cuando las credenciales no coinciden. Ahora el método retorna false limpiamente y dispara la alerta gráfica en la interfaz ("Error: Usuario o contraseña incorrectos.").

Defecto ID	DEF-CU001-02
Caso de Prueba Asociado	CE4 (Nombre de usuario demasiado largo).
Descripción del Defecto	Si el operario escribe un nombre de usuario de más de 20 caracteres, la interfaz gráfica permite enviarlo, lo que provoca un colapso en la base de datos MySQL (Data truncation: Data too long for column 'nombre_usuario').
Severidad	Media (Provoca una caída controlable, pero expone datos internos del motor SQL).
Prioridad de Resolución	Media
Estado	Corregido
Tratamiento / Resolución	Se agregó una validación de longitud del lado del cliente (en la pantalla) antes de invocar al controlador. Si supera los 20

	caracteres, el botón "Ingresar" no se activa y se muestra un texto de advertencia en rojo.

4.4 Evaluación de pruebas

Las pruebas permitieron comprobar que el sistema permite iniciar sesión correctamente cuando se ingresan un nombre de usuario y una contraseña válidos. También se verificó que, ante el ingreso de credenciales incorrectas, el sistema informa adecuadamente el error y permite al usuario volver a ingresar sus datos.

Durante la ejecución de las pruebas se identificaron distintos defectos, los cuales fueron registrados y tratados. Una vez realizadas las correcciones, se repitieron las pruebas afectadas para verificar que los errores hayan sido solucionados y que las modificaciones no afectaran el funcionamiento esperado.

Como resultado final, las pruebas realizadas para el caso de uso CU001 – Iniciar Sesión fueron satisfactorias, cumpliéndose los resultados esperados definidos para los casos de prueba.

5- Prototipo de interfaz

Para el caso de uso “Iniciar Sesión”, al usuario le aparecería la siguiente interfaz:



Para el caso de uso “Registrar Cliente”, al usuario le aparecería la siguiente interfaz:



Para el caso de uso “Registrar Pedido a Proveedor”, al usuario le aparecería la siguiente interfaz:



6- Definición de base de datos para el sistema

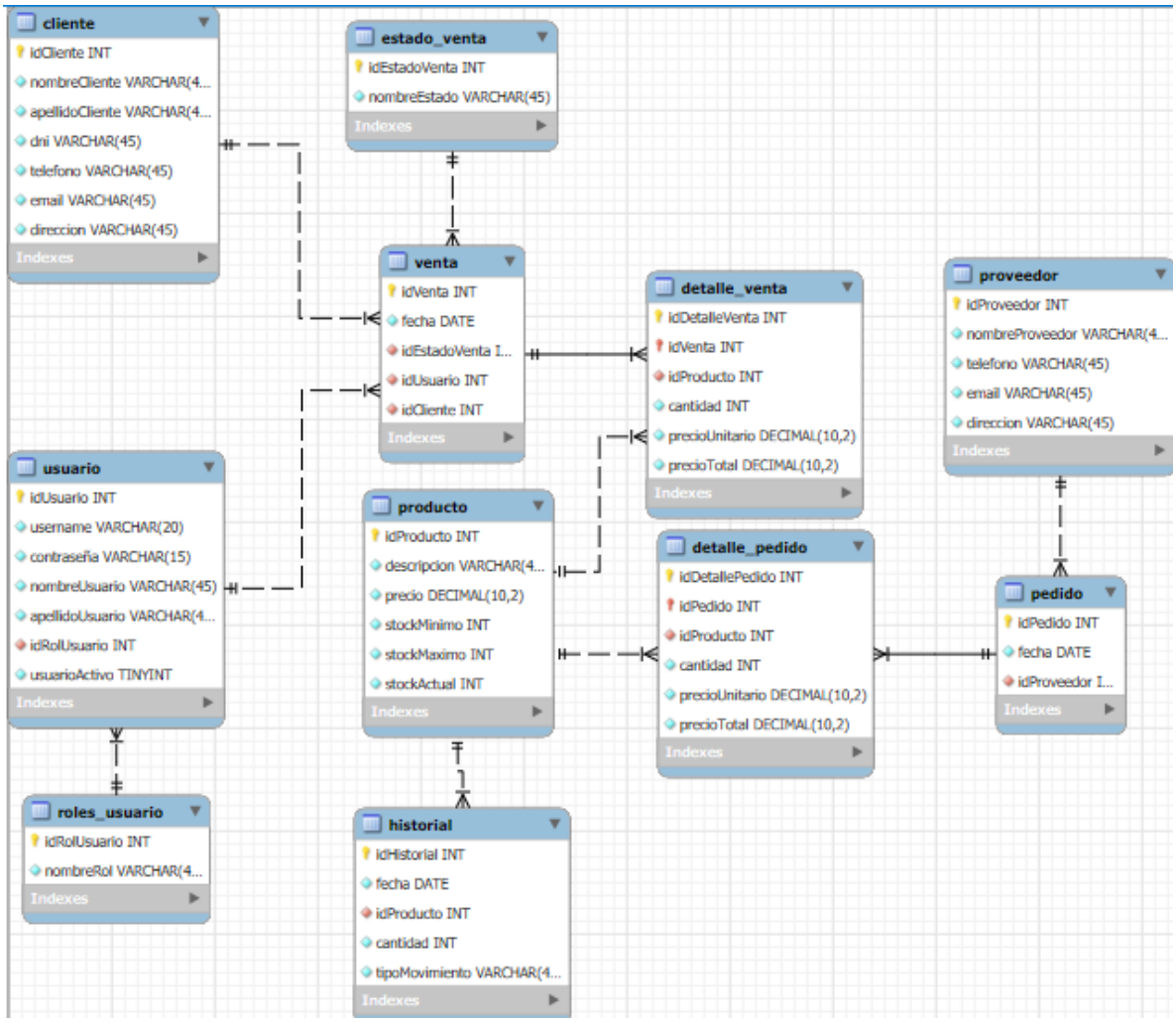
Para el desarrollo del sistema de gestión d stock, se requiere contar con una base de datos relacional. Tal cual como se especifica en los requerimientos no funcionales, en el prototipo se utiliza una base MySQL, que ofrece un buen rendimiento, flexibilidad y escalabilidad. Al considerar que se está trabajando con un prototipo, estas características son muy necesarias para avanzar de forma ordenada con la implementación del proyecto completo.

El uso de una base de datos es fundamental para lograr la persistencia de datos clave, tales como características de los productos, clientes, usuarios, entre otros. Esto es esencial para mantener un historial completo sobre todas las actividades relacionadas al movimiento del stock dentro de la empresa.

Adicionalmente, el almacenamiento de estos datos permite realizar informes en base al historial para analizar las ventas, la entrada y salida de stock, identificar tendencias de productos, etc.

7- Diagrama entidad-relación de la base de datos

El diagrama de entidad-relación (DER) de la base de datos es el siguiente:



8- Creación de las tablas MySQL

Para la base de datos debemos crear las tablas presentes en el diagrama entidad-relación.

Tabla <roles_usuario>

Contiene el identificador único para cada rol y el nombre del rol que puede tener el usuario.

```
-- Tabla de roles de usuario
CREATE TABLE `gestion_stock_distribuidora`.`roles_usuario` (
  `idRolUsuario` INT NOT NULL AUTO_INCREMENT,
  `nombreRol` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idRolUsuario`),
  UNIQUE INDEX `idRolUsuario_UNIQUE` (`idRolUsuario` ASC) VISIBLE
);
```

Tabla <usuario>

Contiene el identificador único del usuario, su nombre de usuario y contraseña, su nombre y apellido, el id del rol y si el usuario esta activo o no.

```
-- Tabla de usuarios
CREATE TABLE `gestion_stock_distribuidora`.`usuario` (
  `idUsuario` INT NOT NULL AUTO_INCREMENT,
  `username` VARCHAR(20) NOT NULL,
  `contraseña` VARCHAR(15) NOT NULL,
  `nombreUsuario` VARCHAR(45) NOT NULL,
  `apellidoUsuario` VARCHAR(45) NOT NULL,
  `idRolUsuario` INT NOT NULL,
  `usuarioActivo` TINYINT NOT NULL,
  PRIMARY KEY (`idUsuario`),
  UNIQUE INDEX `idUsuario_UNIQUE` (`idUsuario` ASC) VISIBLE,
  UNIQUE INDEX `username_UNIQUE` (`username` ASC) VISIBLE,
  CONSTRAINT `fk_usuario_roles_usuario` FOREIGN KEY (`idRolUsuario`) REFERENCES `roles_usuario` (`idRolUsuario`)
);
```

Tabla <cliente>

Contiene el identificador único del cliente, su nombre y apellido, el dni, el teléfono, su dirección de correo electrónico y la dirección en donde vive.

```
-- Tabla de clientes
CREATE TABLE `gestion_stock_distribuidora`.`cliente` (
  `idCliente` INT NOT NULL AUTO_INCREMENT,
  `nombreCliente` VARCHAR(45) NOT NULL,
  `apellidoCliente` VARCHAR(45) NOT NULL,
  `dni` VARCHAR(45) NOT NULL,
  `telefono` VARCHAR(45) NOT NULL,
  `email` VARCHAR(45) NOT NULL,
  `direccion` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idCliente`),
  UNIQUE INDEX `email_UNIQUE` (`email` ASC) VISIBLE,
  UNIQUE INDEX `idCliente_UNIQUE` (`idCliente` ASC) VISIBLE
);
```

Tabla <proveedor>

Contiene el identificador único del proveedor, el nombre, el teléfono, su dirección de correo electrónico y la dirección en donde se encuentra.

```
-- Tabla de proveedores
CREATE TABLE `gestion_stock_distribuidora`.`proveedor` (
  `idProveedor` INT NOT NULL AUTO_INCREMENT,
  `nombreProveedor` VARCHAR(45) NOT NULL,
  `telefono` VARCHAR(45) NOT NULL,
  `email` VARCHAR(45) NOT NULL,
  `direccion` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idProveedor`),
  UNIQUE INDEX `idProveedor_UNIQUE` (`idProveedor` ASC) VISIBLE
);
```

Tabla <producto>

Contiene el identificador único del producto, la descripción, el precio, la cantidad de stock máximo, mínimo y actual.

```
-- Tabla de productos
CREATE TABLE `gestion_stock_distribuidora`.`producto` (
  `idProducto` INT NOT NULL AUTO_INCREMENT,
  `descripcion` VARCHAR(45) NOT NULL,
  `precio` DECIMAL(10,2) NOT NULL,
  `stockMinimo` INT NOT NULL,
  `stockMaximo` INT NOT NULL,
  `stockActual` INT NOT NULL,
  PRIMARY KEY (`idProducto`),
  UNIQUE INDEX `idProducto_UNIQUE` (`idProducto` ASC) VISIBLE
);
```

Tabla <pedido>

Contiene el identificador único del pedido, la fecha en la que se realizo y el id del proveedor al que se realizó el pedido.

```
-- Tabla de pedidos a proveedores
CREATE TABLE `gestion_stock_distribuidora`.`pedido` (
  `idPedido` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATE NOT NULL,
  `idProveedor` INT NOT NULL,
  PRIMARY KEY (`idPedido`),
  UNIQUE INDEX `idPedido_UNIQUE` (`idPedido` ASC) VISIBLE,
  CONSTRAINT `fk_pedido_proveedor` FOREIGN KEY (`idProveedor`) REFERENCES `proveedor` (`idProveedor`)
);
```

Tabla <detalle_pedido>

Contiene el identificador único del detalle del pedido, el id del pedido al que esta relacionado, el id y cantidad del producto que se pidió, el precio unitario del producto y el precio total del pedido.

```sql
-- Tabla del detalle de los pedidos a proveedores
CREATE TABLE `gestion_stock_distribuidora`.`detalle_pedido` (
  `idDetallePedido` INT NOT NULL AUTO_INCREMENT,
  `idPedido` INT NOT NULL,
  `idProducto` INT NOT NULL,
  `cantidad` INT NOT NULL,
  `precioUnitario` DECIMAL(10,2) NOT NULL,
  `precioTotal` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`idDetallePedido`, `idPedido`),
  CONSTRAINT `fk_detalle_pedido_pedido` FOREIGN KEY (`idPedido`) REFERENCES `pedido` (`idPedido`),
  CONSTRAINT `fk_detalle_pedido_producto` FOREIGN KEY (`idProducto`) REFERENCES `producto` (`idProducto`)
);
```

Tabla <historial>

Contiene el identificador único del historial, la fecha en que se realizo el movimiento, el id del producto que se modificó, la cantidad que se modifico y el tipo de movimiento que se realizó (Ingreso/Egreso).

```sql
-- Tabla del historial del movimiento del stock
CREATE TABLE `gestion_stock_distribuidora`.`historial` (
  `idHistorial` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATE NOT NULL,
  `idProducto` INT NOT NULL,
  `cantidad` INT NOT NULL,
  `tipoMovimiento` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idHistorial`),
  UNIQUE INDEX `idHistorial_UNIQUE` (`idHistorial` ASC) VISIBLE,
  CONSTRAINT `fk_historial_producto` FOREIGN KEY (`idProducto`) REFERENCES `producto` (`idProducto`)
);
```

Tabla <estado_venta>

Contiene el identificador único de los posibles estados de una venta, y el nombre de cada estado.

```sql
-- Tabla de los posibles estados de una venta de productos
CREATE TABLE `gestion_stock_distribuidora`.`estado_venta` (
  `idEstadoVenta` INT NOT NULL AUTO_INCREMENT,
  `nombreEstado` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idEstadoVenta`),
  UNIQUE INDEX `idEstadoVenta_UNIQUE` (`idEstadoVenta` ASC) VISIBLE
);
```

Tabla <venta>

Contiene el identificador único de la venta de un producto a un cliente, la fecha en la que se realizó y el id del estado en el que se encuentra la venta, el id del usuario que realizó la venta y el id del cliente al que se le vendió.

```sql
-- Tabla de venta de productos
CREATE TABLE `gestion_stock_distribuidora`.`venta` (
  `idVenta` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATE NOT NULL,
  `idEstadoVenta` INT NOT NULL,
  `idUsuario` INT NOT NULL,
  `idCliente` INT NOT NULL,
  PRIMARY KEY (`idVenta`),
  UNIQUE INDEX `idVenta_UNIQUE` (`idVenta` ASC) VISIBLE,
  CONSTRAINT `fk_venta_estado_venta` FOREIGN KEY (`idEstadoVenta`) REFERENCES `estado_venta` (`idEstadoVenta`),
  CONSTRAINT `fk_venta_usuario` FOREIGN KEY (`idUsuario`) REFERENCES `usuario` (`idUsuario`),
  CONSTRAINT `fk_venta_cliente` FOREIGN KEY (`idCliente`) REFERENCES `cliente` (`idCliente`)
);
```

Tabla <detalle_venta>

Contiene el identificador único del detalle de una venta, el id de la venta, el id del producto que se vendió, la cantidad, el precio unitario y el precio total.

```sql
-- Tabla del detalle de las ventas
CREATE TABLE `gestion_stock_distribuidora`.`detalle_venta` (
  `idDetalleVenta` INT NOT NULL AUTO_INCREMENT,
  `idVenta` INT NOT NULL,
  `idProducto` INT NOT NULL,
  `cantidad` INT NOT NULL,
  `precioUnitario` DECIMAL(10,2) NOT NULL,
  `precioTotal` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`idDetalleVenta`, `idVenta`),
  CONSTRAINT `fk_detalle_venta_venta` FOREIGN KEY (`idVenta`) REFERENCES `venta` (`idVenta`),
  CONSTRAINT `fk_detalle_venta_producto` FOREIGN KEY (`idProducto`) REFERENCES `producto` (`idProducto`)
);
```

9- Inserción, consulta y borrado de registro.

Insertar datos de prueba:

Tabla <roles_usuario>

```sql
INSERT INTO roles_usuario (nombreRol)
VALUES
    ('Administrador'),
    ('Encargado de almacen');
```

Tabla <usuario>

```sql
INSERT INTO usuario (username, contraseña, nombreUsuario, apellidoUsuario, idRolUsuario, usuarioActivo)
VALUES
    ('Admin123', 'clave123', 'Juan', 'Perez', 1, true),
    ('Pepe', 'clave222', 'Pedro', 'Diaz', 2, true),
    ('Felipe', 'clave333', 'Felipe', 'Belli', 2, true);
```

Tabla <cliente>

```
INSERT INTO cliente (nombreCliente, apellidoCliente, dni, telefono, email, direccion)
VALUES
    ('Francisco', 'Rojas', '46499888', '3413876492', 'franrj@gmail.com', 'San Martín 820, Rosario, Santa Fe'),
    ('Valentina', 'Gómez', '20948274', '3413646284', 'valen.gomez.01@yahoo.com', 'Av. Colón 2341, Rosario, Santa Fe'),
    ('Mateo', 'Fernandez', '28983250', '3413092326', 'mateo.fernandez@hotmail.com', 'Av. Pellegrini 1180, Rosario, Santa Fe');
```

Tabla <proveedor>

```
INSERT INTO proveedor (nombreProveedor, telefono, email, direccion)
VALUES
    ('Distribuidora La Cosecha S.A.', '341876543', 'contacto@lacosecha.com.ar', 'Av. Circunvalación 1250, Rosario, Santa Fe'),
    ('Alimentos del Centro S.R.L.', '341939494', 'ventas@alimentosdelcentro.com.ar', 'Av. Colón 1840, Rosario, Santa Fe'),
    ('Frigorífica y Alimentos Andinos S.A.', '341098772', 'info@alimentandinos.com.ar', 'San Martín 920, Rosario, Santa Fe');
```

Tabla <producto>

```
INSERT INTO producto (descripcion, precio, stockMinimo, stockMaximo, stockActual)
VALUES
    ('Arroz largo fino 1kg', 1500.00, 20, 100, 40),
    ('Aceite de oliva 500ml', 4500.00, 10, 60, 30),
    ('Atún en lata 170g', 2800.00, 10, 50, 20),
    ('Lentejas secas 500g', 1800.00, 15, 80, 20);
```

Tabla <pedido>

```
INSERT INTO pedido (fecha, idProveedor)
VALUES
    ('2026-09-10', 1),
    ('2026-09-10', 2),
    ('2026-09-20', 1);
```

Tabla <detalle_pedido>

```
INSERT INTO detalle_pedido (idPedido, idProducto, cantidad, precioUnitario, precioTotal)
VALUES
    (1, 1, 60, 1500, 90000.00),
    (1, 2, 30, 4500, 135000.00),

    (2, 3, 30, 2800.00, 84000.0),
    (2, 4, 40, 1800.00, 72000.00),

    (3, 4, 20, 1800.00, 36000.00);
```

Tabla <estado_venta>

```
INSERT INTO estado_venta (nombreEstado)
VALUES
    ('Pendiente'),
    ('Confirmado'),
    ('Anulado');
```

Tabla <venta>

```
INSERT INTO venta (fecha, idEstadoVenta, idUsuario, idCliente)
VALUE
    ('2026-09-05', 2, 2, 1),
    ('2026-09-19', 1, 2, 2),
    ('2026-09-15', 2, 3, 1);
```

Tabla <detalle_venta>

```
INSERT INTO detalle_venta (idVenta, idProducto, cantidad, precioUnitario, precioTotal)
VALUE
    (1, 1, 10, 1500.00, 15000.00),
    (1, 2, 1, 4500.00, 4500.00),

    (2, 4, 7, 1800.00, 12600.00),

    (3, 2, 2, 4500.00, 9000.00),
    (3, 3, 5, 2800.00, 14000.00);
```

Tabla <historial>

```
INSERT INTO historial (fecha, idProducto, cantidad, tipoMovimiento)
VALUES
    ('2026-09-10', 1, 60, 'Ingreso'),
    ('2026-09-10', 2, 30, 'Ingreso'),
    ('2026-09-10', 3, 30, 'Ingreso'),
    ('2026-09-10', 4, 40, 'Ingreso'),
    ('2026-09-20', 4, 20, 'Ingreso'),

    ('2026-09-05', 1, 10, 'Egreso'),
    ('2026-09-05', 2, 1, 'Egreso'),
    ('2026-09-19', 4, 7, 'Egreso'),
    ('2026-09-15', 2, 2, 'Egreso'),
    ('2026-09-15', 3, 5, 'Egreso');
```

Consulta de datos de prueba

Consultar Detalle de Ventas (Con información del Cliente y Producto)

```sql
-- Consultar Detalle de Ventas (Con información del Cliente y Producto)
SELECT
    VE.fecha AS 'Fecha Venta',
    CL.nombreCliente AS 'Nombre Cliente',
    CL.apellidoCliente AS 'Apellido Cliente',
    PR.descripcion AS 'Producto',
    DV.cantidad AS 'Cantidad',
    DV.precioUnitario AS 'Precio Unitario',
    DV.precioTotal AS 'Precio Total',
    US.username AS 'Vendedor'
FROM
    venta VE,
    cliente CL,
    detalle_venta DV,
    producto PR,
    usuario US
WHERE
    VE.idCliente = CL.idCliente
    AND VE.idVenta = DV.idVenta
    AND DV.idProducto = PR.idProducto
    AND VE.idUsuario = US.idUsuario;
```

Fecha Venta	Nombre Cliente	Apellido Cliente	Producto	Cantidad	Precio Unitario	Precio Total	Vendedor
2026-09-05	Francisco	Rojas	Arroz largo fino 1kg	10	1500.00	15000.00	Pepe
2026-09-05	Francisco	Rojas	Aceite de oliva 500ml	1	4500.00	4500.00	Pepe
2026-09-19	Valentina	Gómez	Lentejas secas 500g	7	1800.00	12600.00	Pepe
2026-09-15	Francisco	Rojas	Aceite de oliva 500ml	2	4500.00	9000.00	Felipe
2026-09-15	Francisco	Rojas	Atún en lata 170g	5	2800.00	14000.00	Felipe

Consultar Detalle de Pedidos (A Proveedores)

```sql
-- Consultar Detalle de Pedidos (A Proveedores)
SELECT
    PE.fecha AS 'Fecha Pedido',
    PV.nombreProveedor AS 'Proveedor',
    PR.descripcion AS 'Producto',
    DP.cantidad AS 'Cantidad',
    DP.precioUnitario AS 'Costo Unitario',
    DP.precioTotal AS 'Costo Total'
FROM
    pedido PE,
    proveedor PV,
    detalle_pedido DP,
    producto PR
WHERE
    PE.idProveedor = PV.idProveedor
    AND PE.idPedido = DP.idPedido
    AND DP.idProducto = PR.idProducto;
```

Fecha Pedido	Proveedor	Producto	Cantidad	Costo Unitario	Costo Total
2026-09-10	Distribuidora La Cosecha S.A.	Arroz largo fino 1kg	60	1500.00	90000.00
2026-09-10	Distribuidora La Cosecha S.A.	Aceite de oliva 500ml	30	4500.00	135000.00
2026-09-10	Alimentos del Centro S.R.L.	Atún en lata 170g	30	2800.00	84000.00
2026-09-10	Alimentos del Centro S.R.L.	Lentejas secas 500g	40	1800.00	72000.00
2026-09-20	Distribuidora La Cosecha S.A.	Lentejas secas 500g	20	1800.00	36000.00

Consultar Historial de Movimientos de Stock

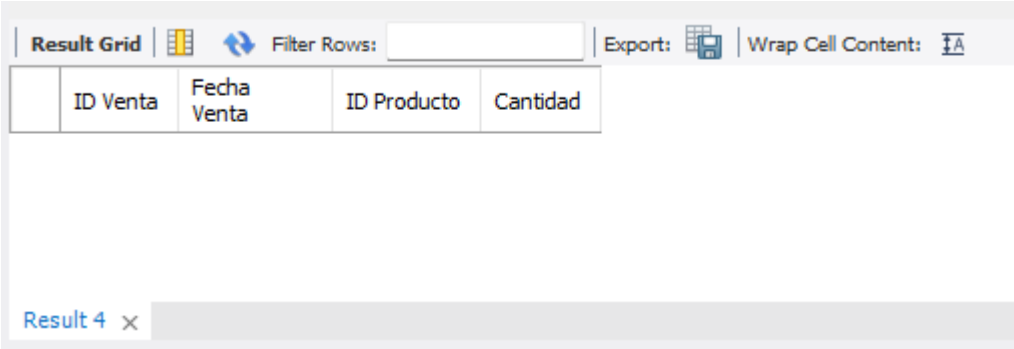
```sql
-- Consultar Historial de Movimientos de Stock
SELECT
    HI.fecha AS 'Fecha Movimiento',
    PR.descripcion AS 'Producto',
    HI.cantidad AS 'Cantidad',
    HI.tipoMovimiento AS 'Tipo de Movimiento'
FROM
    historial HI,
    producto PR
WHERE
    HI.idProducto = PR.idProducto
ORDER BY
    HI.fecha DESC;
```

Fecha Movimiento	Producto	Cantidad	Tipo de Movimiento
2026-09-20	Lentejas secas 500g	20	Ingreso
2026-09-19	Lentejas secas 500g	7	Egreso
2026-09-15	Aceite de oliva 500ml	2	Egreso
2026-09-15	Atún en lata 170g	5	Egreso
2026-09-10	Arroz largo fino 1kg	60	Ingreso
2026-09-10	Aceite de oliva 500ml	30	Ingreso
2026-09-10	Atún en lata 170g	30	Ingreso
2026-09-10	Lentejas secas 500g	40	Ingreso
2026-09-05	Arroz largo fino 1kg	10	Egreso
2026-09-05	Aceite de oliva 500ml	1	Egreso

Borrado de datos de prueba

Eliminar una Venta Específica

```sql
-- Eliminar una Venta Especitica
-- Paso 1: Eliminar el detalle de los productos asociados a la venta
DELETE DV
FROM detalle_venta DV
WHERE DV.idVenta = 2;
-- Paso 2: Eliminar la venta principal
DELETE VE
FROM venta VE
WHERE VE.idVenta = 2;
-- Comprobacion de la eliminación de la Venta (idVenta = 2)
SELECT
    VE.idVenta AS 'ID Venta',
    VE.fecha AS 'Fecha Venta',
    DV.idProducto AS 'ID Producto',
    DV.cantidad AS 'Cantidad'
FROM
    venta VE
LEFT JOIN
    detalle_venta DV ON VE.idVenta = DV.idVenta
WHERE
    VE.idVenta = 2;
```



Borrado lógico de usuario

```
-- Borrado lógico de usuario
UPDATE usuario US
SET US.usuarioActivo = 0
WHERE US.idUsuario = 3;
-- Comprobar el estado del Usuario (Borrado Lógico)
SELECT
    US.idUsuario AS 'ID Usuario',
    US.username AS 'Nombre de Usuario',
    US.usuarioActivo AS 'Estado Activo (1=Sí, 0=No)'
FROM
    usuario US
WHERE
    US.idUsuario = 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID Usuario	Nombre de Usuario	Estado Activo (1=Sí, 0=No)
3	Felipe	0

10- Definiciones de comunicación

Infraestructura física

En base al RNF05, toda la infraestructura informática se despliega de manera local en las oficinas centrales de la distribuidora.

- **Red de Área Local (LAN):** Se implementa una topología de red en estrella mediante un Switch Administrable de 24 puertos Gigabit Ethernet. Esto garantiza el ancho de banda necesario para soportar la concurrencia de operarios de forma simultánea sin degradar el rendimiento del sistema (RNF08).

- **Segmentación de Red**: Se configuran VLANs para aislar el tráfico de datos:
    - **VLAN Administración/Depósito**: Para las computadoras de los operarios de stock y facturistas.
    - **VLAN Servidores**: Acceso restringido únicamente para el Servidor de Aplicación y el Servidor de Base de Datos, previniendo intrusiones.
- **Hardware de los Servidores**: Servidores en formato Rack con fuentes de alimentación redundantes, conectados directamente a una UPS (Sistema de Energía Ininterrumpida) para asegurar la tolerancia a fallos y la disponibilidad 7x24 (RNF03).

Enlace de datos

Define cómo se empaquetan los bits y cómo se controla el acceso al medio físico de transmisión para evitar colisiones y pérdida de datos en el depósito.

- **Cableado Estructurado**: Se utiliza cable de red de cobre UTP Categoría 6 para todas las conexiones cableadas de las computadoras fijas, garantizando velocidades de hasta 1 Gbps y reduciendo la interferencia electromagnética de las cámaras de frío de la distribuidora.
- **Estándar IEEE 802.3 (Ethernet):** Protocolo base para la comunicación cableada en la LAN, encargado del direccionamiento físico por medio de direcciones MAC y la detección de errores mediante secuencias de verificación de tramas (FCS).

Protocolos y estándares de comunicación

Se utilizan los protocolos **HTTP/HTTPS (Puerto 8080 estándar de Apache Tomcat)** para el intercambio de información entre la interfaz y el controlador, por ejemplo, para los formularios de iniciar sesión y registrar cliente, empaquetando los datos de entrada y realizando peticiones tipo POST o GET hacia el controlador.

Para la conexión a la base de datos se utiliza el protocolo **JDBC (Java Database Connectivity)** sobre el estándar **TCP/IP** (Puerto por defecto 3306 de MySQL). Las clases DAO (Data Access Object) abren un canal de comunicación directo hacia la base de datos. Las consultas SQL (INSERT, UPDATE, SELECT) viajan encapsuladas en tramas TCP y MySQL retorna el número de filas afectadas o conjuntos de registros como respuesta técnica.

Para la facturación electrónica se utiliza el protocolo **HTTPS** (Puerto 443) empleando **Web Services (REST o SOAP)**, el cual cifra mensajes mediante el formato JSON o XML. El sistema consume la API externa enviando el detalle de la venta y recibe la autorización fiscal del comprobante antes de imprimir la factura.